# Coding Sample: Escaping Inequality in India

Rishav Roy

What follows is the 138-page code appendix to my independent paper and writing sample, "Escaping Inequality in India: The Role of English-Medium Instruction."

Due to the length of this code, the time-constrained reviewer is encouraged to prioritize these 47 pages, excerpted from the code chunks listed below them:

14-16. Scalable **import, cleaning, and merging** of multi-file census and geospatial data.

- Chunk 4: Import key data files

31-32. **QA checks, disambiguation of identifiers** for district-level crosswalk.

- Chunk 6: Match districts: Construct district tracking dfs

50-52. **Testing for systematic missingness** using the Benjamini–Hochberg procedure and parallelized logistic regressions.

- Chunk 8: Participation model: Are NAs randomly distributed

55-57. **Survey-weighted enrollment probit estimation** and average marginal effects derivation under a complex survey design. The enrollment model is reported as a stand-alone descriptive selection model; district-level inverse-Mills-ratio aggregation is not part of the active analysis.

- Chunk 9: Participation model: Survey-weighted estimation
- Chunk 10: Participation model: Derive AME

80-88. A reviewed **district-harmonization map contract** with explicit many-to-many QA, while source attachment and candidate diagnostics remain separated from the public crosswalk authority.

- Chunk 16: Match districts: Test joining methods
- Chunk 17: Match districts: Define joining functions

105-107. Reusable **IV specification and estimation pipeline** with clustered inference, plus contiguity-based spatial weights and setup for spatially lagged models.

- Chunk 24: Geospatial: Neighbor list construction
- Chunk 25: 2SLS: Controls and 2SLS formula
- Chunk 26: 2SLS: Baseline 2SLS models

111-112. **Robustness checks and diagnostics** for multicollinearity, spatial autocorrelation.

- Chunk 28: Multicollinearity check
- Chunk 29: Geospatial: Spatial autocorrelation tests

117-138. **Automated generation of publication-quality figures and tables** (including both summary statistics and regression outputs).

- Chunks 31-40: [Output for paper]

## Scalable import, cleaning, and merging of Census/geospatial data

Source: `R/io/read_inputs.R`

```r
#' read nss 2007 education
#'
read_nss_2007_education <- function(paths) {
  read_manifest_group(paths, "nss_2007_education")
}

#' read nss 2007 consumption
#'
read_nss_2007_consumption <- function(paths) {
  read_manifest_group(paths, "nss_2007_consumption")
}

#' read nss 2017 education
#'
read_nss_2017_education <- function(paths) {
  read_manifest_group(paths, "nss_2017_education")
}

#' read census 2001 mother tongue
#'
read_census_2001_mother_tongue <- function(paths) {
  read_manifest_group(paths, "census_2001_mother_tongue")
}

#' read district boundaries 2020
#'
read_district_boundaries_2020 <- function(paths) {
  rows <- require_manifest_files(paths, "district_boundaries_2020")
  shp <- rows[tolower(rows$file_type) == "shp", , drop = FALSE]
  if (!nrow(shp)) stop("The district-boundary manifest includes shapefile sidecars but no
   ↪  .shp row.", call. = FALSE)
  need_pkg("sf", "district boundaries")
  sf::st_read(shp$absolute_path[[1]], quiet = TRUE)
}

#' read district change sources
#'
read_district_change_sources <- function(paths) {
  read_manifest_group(paths, "district_changes")
}

#' list ilo figure paths
#'
list_ilo_figure_paths <- function(paths) {
  rows <- require_manifest_files(paths, "ilo_figures")
  stats::setNames(rows$absolute_path, rows$file_id)
}

#' Read the headerless 1961-2001 district carve-out source
#'
#' The source has five data columns and no header row. Reading it with the
#' ordinary CSV default would consume the first Anantapur observation as names.
```

```r
read_district_carveouts <- function(path) {
  out <- utils::read.csv(
    path,
    header = FALSE,
    col.names = c("district_1991", "pop_1991", "district_2001", "pct_01in91", "pct_91in01"),
    stringsAsFactors = FALSE,
    na.strings = c("", "NA"),
    check.names = FALSE
  )
  fill_down <- function(x) {
    if (!length(x)) return(x)
    for (i in seq_along(x)) {
      missing <- is.na(x[[i]]) || (is.character(x) && !nzchar(trimws(x[[i]])))
      if (missing && i > 1L) x[[i]] <- x[[i - 1L]]
    }
    x
  }
  out$district_1991 <- fill_down(out$district_1991)
  out$pop_1991 <- num(gsub(",", "", fill_down(out$pop_1991), fixed = TRUE))
  out$pct_01in91 <- num(out$pct_01in91)
  out$pct_91in01 <- num(out$pct_91in01)
  out
}

#' read one manifest row
#'
#' @return Reader output for raw data files, or a validated path for sidecars/assets.
read_by_manifest_row <- function(row) {
  path <- row$absolute_path[[1]]
  reader <- as.character(row$reader_function[[1]] %||% "")
  if (identical(reader, "read_district_carveouts")) {
    return(read_district_carveouts(path))
  }
  file_type <- tolower(row$file_type[[1]])
  switch(
    file_type,
    sav = read_sav_short(path),
    csv = read_csv_short(path),
    xls = read_excel_short(path),
    xlsx = read_excel_short(path),
    ods = read_ods_short(path),
    shp = {
      need_pkg("sf", "shapefiles")
      sf::st_read(path, quiet = TRUE)
    },
    shx = path,
    dbf = path,
    prj = path,
    png = path,
    stop("No reader implemented for ", file_type, call. = FALSE)
  )
}

#' read all manifest rows for a source
#'
```

```r
#' @return Named list of reader outputs keyed by manifest file_id.
read_manifest_group <- function(paths, source_id) {
  rows <- require_manifest_files(paths, source_id)
  stats::setNames(lapply(seq_len(nrow(rows)), function(i) read_by_manifest_row(rows[i, ])),
  ↪ rows$file_id)
}
```

## QA and identifier-disambiguation checks

Source: `R/districts/build_district_tracker.R`

```r
#' build district tracker
#'
build_district_tracker <- function(raw_district_changes) {
  source_aliases <- list(
    alluvial = c("alluvial", "district_changes_alluvial"),
    carveouts_renamings = c("carveouts_renamings", "district_changes_carveouts"),
    new_districts_created = c("new_districts_created", "district_changes_new_districts"),
    name_changes = c("name_changes", "district_changes_name_changes"),
    district_splits = c("district_splits", "district_changes_splits"),
    india_district_tracker = c("india_district_tracker", "tracker",
    ↪ "district_changes_tracker")
  )
  source_value <- function(id) {
    hit <- source_aliases[[id]][source_aliases[[id]] %in% names(raw_district_changes)]
    if (length(hit)) raw_district_changes[[hit[[1]]]] else data.frame()
  }
  parsed <- list(
    alluvial = parse_alluvial_district_changes(source_value("alluvial")),
    carveouts_renamings = parse_carveouts_renamings(source_value("carveouts_renamings")),
    new_districts_created =
    ↪ parse_new_districts_created(source_value("new_districts_created")),
    name_changes = parse_name_changes(source_value("name_changes")),
    district_splits = parse_district_splits(source_value("district_splits")),
    india_district_tracker =
    ↪ parse_india_district_tracker(source_value("india_district_tracker"))
  )
  consumed <- unique(unlist(source_aliases, use.names = FALSE))
  extras <- setdiff(names(raw_district_changes), consumed)
  for (nm in extras) parsed[[nm]] <-
  ↪ parse_district_change_source(raw_district_changes[[nm]], source_type = nm)
  standardize_tracker_names(standardize_tracker_years(combine_district_tracker_sources
  ↪ (parsed)))
}

#' parse alluvial district changes
#'
parse_alluvial_district_changes <- function(x) {
  x <- safe_df(x)
  year_state <- grep("^[0-9]{4}.*state", canon(names(x)), value = FALSE)
  years <- suppressWarnings(as.integer(sub("^([0-9]{4}).*$", "\\1",
  ↪ canon(names(x)[year_state]))))
  years <- sort(unique(years[is.finite(years)]))
  if (length(years) < 2L) return(parse_district_change_source(x, source_type = "alluvial"))
  parse_wide_year_lineages(x, years, source_type = "alluvial")
```

```r
}

#' parse india district tracker
#'
parse_india_district_tracker <- function(x) {
  x <- safe_df(x)
  years <- suppressWarnings(as.integer(names(x)[grepl("^[0-9]{4}$", names(x))]))
  years <- sort(unique(years[is.finite(years)]))
  if (length(years) < 2L) return(parse_district_change_source(x, source_type =
   ↪  "india_district_tracker"))
  parse_tracker_year_triplets(x, years)
}

#' parse carveouts renamings
#'
parse_carveouts_renamings <- function(x) {
  x <- safe_df(x)
  required <- c("district_1991", "pop_1991", "district_2001", "pct_01in91", "pct_91in01")
  if (!all(required %in% names(x))) {
    return(parse_district_change_source(x, source_type = "carveouts_renamings"))
  }
  data.frame(
    source_type = "carveouts_renamings",
    source_state_raw = NA_character_,
    source_district_raw = plain_chr(x$district_1991),
    target_state_raw = NA_character_,
    target_district_raw = plain_chr(x$district_2001),
    source_year_raw = 1991L,
    target_year_raw = 2001L,
    change_type = "population_transfer_1991_2001",
    event_year = 2001L,
    source_population = num(gsub(",", "", plain_chr(x$pop_1991), fixed = TRUE)),
    source_share_to_target = num(x$pct_01in91) / 100,
    target_share_from_source = num(x$pct_91in01) / 100,
    stringsAsFactors = FALSE
  )
}

#' parse new districts created
#'
parse_new_districts_created <- function(x) {
  x <- safe_df(x)
  if (!all(c("State/UT", "Old districts", "New District") %in% names(x))) {
    return(parse_district_change_source(x, source_type = "new_districts_created"))
  }
  data.frame(
    source_type = "new_districts_created",
    source_state_raw = plain_chr(x[["State/UT"]]),
    source_district_raw = plain_chr(x[["Old districts"]]),
    target_state_raw = plain_chr(x[["State/UT"]]),
    target_district_raw = plain_chr(x[["New District"]]),
    source_year_raw = suppressWarnings(as.integer(x$Year)) - 1L,
    target_year_raw = suppressWarnings(as.integer(x$Year)),
    change_type = "new_district",
    event_year = suppressWarnings(as.integer(x$Year)),
```

```r
    decade = plain_chr(x$Decade),
    stringsAsFactors = FALSE
  )
}

#' parse name changes
#'
parse_name_changes <- function(x) {
  x <- safe_df(x)
  if (!all(c("State/UT", "Old Name", "New Name") %in% names(x))) {
    return(parse_district_change_source(x, source_type = "name_changes"))
  }
  decade_end <- suppressWarnings(as.integer(sub(".*-", "", plain_chr(x$Decade))))
  data.frame(
    source_type = "name_changes",
    source_state_raw = plain_chr(x[["State/UT"]]),
    source_district_raw = plain_chr(x[["Old Name"]]),
    target_state_raw = plain_chr(x[["State/UT"]]),
    target_district_raw = plain_chr(x[["New Name"]]),
    source_year_raw = NA_integer_,
    target_year_raw = decade_end,
    change_type = paste0("name_change:", canon(x$Type)),
    event_year = NA_integer_,
    decade = plain_chr(x$Decade),
    stringsAsFactors = FALSE
  )
}

#' parse district splits
#'
parse_district_splits <- function(x) {
  x <- safe_df(x)
  if (!all(c("State/UT", "District-Before", "District-After") %in% names(x))) {
    return(parse_district_change_source(x, source_type = "district_splits"))
  }
  year <- suppressWarnings(as.integer(x$Year))
  data.frame(
    source_type = "district_splits",
    source_state_raw = plain_chr(x[["State/UT"]]),
    source_district_raw = plain_chr(x[["District-Before"]]),
    target_state_raw = plain_chr(x[["State/UT"]]),
    target_district_raw = plain_chr(x[["District-After"]]),
    source_year_raw = year - 1L,
    target_year_raw = year,
    change_type = "split_or_carveout",
    event_year = year,
    decade = plain_chr(x$Decade),
    stringsAsFactors = FALSE
  )
}


find_year_field <- function(x, year, kind) {
  keys <- canon(names(x))
  pattern <- paste0("^", year, " ", kind)
```

```r
  hit <- which(grepl(pattern, keys))
  if (length(hit)) names(x)[hit[[1]]] else NULL
}

parse_wide_year_lineages <- function(x, years, source_type) {
  rows <- lapply(seq_len(length(years) - 1L), function(i) {
    source_year <- years[[i]]
    target_year <- years[[i + 1L]]
    ss <- find_year_field(x, source_year, "state")
    sd <- find_year_field(x, source_year, "district")
    ts <- find_year_field(x, target_year, "state")
    td <- find_year_field(x, target_year, "district")
    if (any(vapply(list(ss, sd, ts, td), is.null, logical(1)))) return(data.frame())
    out <- data.frame(
      source_type = source_type,
      source_state_raw = plain_chr(x[[ss]]),
      source_district_raw = plain_chr(x[[sd]]),
      target_state_raw = plain_chr(x[[ts]]),
      target_district_raw = plain_chr(x[[td]]),
      source_year_raw = source_year,
      target_year_raw = target_year,
      stringsAsFactors = FALSE
    )
    source_pair <- paste(canonicalize_state_name(out$source_state_raw),
 ↪  canon(out$source_district_raw), sep = "__")
    target_pair <- paste(canonicalize_state_name(out$target_state_raw),
 ↪  canon(out$target_district_raw), sep = "__")
    out$change_type <- ifelse(source_pair == target_pair, "continuity", "lineage_candidate")
    out$.source_wide_row <- seq_len(nrow(x))
    keep <- !is.na(out$source_district_raw) & nzchar(trimws(out$source_district_raw)) &
      !is.na(out$target_district_raw) & nzchar(trimws(out$target_district_raw))
    out[keep, , drop = FALSE]
  })
  safe_bind_rows(rows)
}

parse_tracker_year_triplets <- function(x, years) {
  # The Jaacks sheet has state, district, and code columns in repeating
  # three-column year blocks, with the first data row containing field labels.
  if (nrow(x) && any(canon(unlist(x[1, ], use.names = FALSE)) == "statename")) x <- x[-1, ,
   ↪  drop = FALSE]
  rows <- lapply(seq_len(length(years) - 1L), function(i) {
    source_year <- years[[i]]
    target_year <- years[[i + 1L]]
    source_col <- match(as.character(source_year), names(x))
    target_col <- match(as.character(target_year), names(x))
    if (!is.finite(source_col) || !is.finite(target_col)) return(data.frame())
    out <- data.frame(
      source_type = "india_district_tracker",
      source_state_raw = plain_chr(x[[source_col]]),
      source_district_raw = plain_chr(x[[source_col + 1L]]),
      source_code_raw = plain_chr(x[[source_col + 2L]]),
      target_state_raw = plain_chr(x[[target_col]]),
      target_district_raw = plain_chr(x[[target_col + 1L]]),
      target_code_raw = plain_chr(x[[target_col + 2L]]),
```

```r
      source_year_raw = source_year,
      target_year_raw = target_year,
      stringsAsFactors = FALSE
    )
    source_pair <- paste(canonicalize_state_name(out$source_state_raw),
  ↪ canon(out$source_district_raw), sep = "__")
    target_pair <- paste(canonicalize_state_name(out$target_state_raw),
  ↪ canon(out$target_district_raw), sep = "__")
    out$change_type <- ifelse(source_pair == target_pair, "continuity",
  ↪ "annual_lineage_candidate")
    out$.source_wide_row <- seq_len(nrow(x))
    keep <- !is.na(out$source_district_raw) & nzchar(trimws(out$source_district_raw)) &
      !is.na(out$target_district_raw) & nzchar(trimws(out$target_district_raw))
    out[keep, , drop = FALSE]
  })
  safe_bind_rows(rows)
}

parse_district_change_source <- function(x, source_type) {
  x <- safe_df(x)
  if (!nrow(x)) return(data.frame(source_type = character(), stringsAsFactors = FALSE))
  x$source_type <- source_type
  x$source_state_raw <- first_present_value(x, c("state", "State", "state_raw",
  ↪ "state_from", "state_01", "state_07", "state_08", "state_17", "state_18", "state_20"))
  x$source_district_raw <- first_present_value(x, c("district", "District", "district_raw",
  ↪ "district_from", "district_01", "district_07", "district_08", "district_17",
  ↪ "district_18", "district_20", "district_name"))
  x$target_state_raw <- first_present_value(x, c("state_to", "new_state", "target_state",
  ↪ "state_20", "state_18", "state_17", "state_08", "state_07", "state"))
  x$target_district_raw <- first_present_value(x, c("district_to", "new_district",
  ↪ "target_district", "district_20", "district_18", "district_17", "district_08",
  ↪ "district_07", "district"))
  x$source_year_raw <- first_present_value(x, c("source_year", "year_from", "from_year",
  ↪ "start_year", "year"))
  x$target_year_raw <- first_present_value(x, c("target_year", "year_to", "to_year",
  ↪ "end_year", "year"))
  x$change_type <- first_present_value(x, c("change_type", "type", "event_type", "status"))
  x
}

first_present_value <- function(df, candidates) {
  hit <- first_col(df, candidates)
  if (is.null(hit)) return(rep(NA_character_, nrow(df)))
  as.character(df[[hit]])
}

#' combine district tracker sources
#'
combine_district_tracker_sources <- function(raw_district_changes) {
  out <- safe_bind_rows(lapply(names(raw_district_changes), function(name) {
    x <- safe_df(raw_district_changes[[name]])
    if (!nrow(x)) return(data.frame())
    x$source_file_id <- if ("source_file_id" %in% names(x)) x$source_file_id else name
    if (!"source_type" %in% names(x)) x$source_type <- name
    x
```

```r
  }))
  if (nrow(out)) out$.row_in_source <- ave(seq_len(nrow(out)), out$source_file_id, FUN =
  ↪  seq_along)
  out
}

#' standardize tracker years
#'
standardize_tracker_years <- function(tracker) {
  tracker <- safe_df(tracker)
  if (!nrow(tracker)) return(tracker)
  if ("source_year_raw" %in% names(tracker) && !"source_year" %in% names(tracker))
  ↪  tracker$source_year <- suppressWarnings(as.integer(tracker$source_year_raw))
  if ("target_year_raw" %in% names(tracker) && !"target_year" %in% names(tracker))
  ↪  tracker$target_year <- suppressWarnings(as.integer(tracker$target_year_raw))
  tracker
}

#' standardize tracker names
#'
standardize_tracker_names <- function(tracker) {
  tracker <- safe_df(tracker)
  if (!nrow(tracker)) return(tracker)
  if ("source_state_raw" %in% names(tracker)) tracker$source_state_key <-
  ↪  canonicalize_state_name(tracker$source_state_raw)
  if ("source_district_raw" %in% names(tracker)) tracker$source_district_key <-
  ↪  canon(tracker$source_district_raw)
  if ("target_state_raw" %in% names(tracker)) tracker$target_state_key <-
  ↪  canonicalize_state_name(tracker$target_state_raw)
  if ("target_district_raw" %in% names(tracker)) tracker$target_district_key <-
  ↪  canon(tracker$target_district_raw)
  tracker
}
```

### Parallelized missingness diagnostics with BH correction

Source: R/selection/diagnose_missingness.R

```r
missingness_variables <- function(selection_data) {
  df <- as.data.frame(selection_data, stringsAsFactors = FALSE)
  all_child_missing_vars <- c("DIST_FROM_NEAREST_PRIMARY_CLASS",
↪  "dmean_num_ENROLLMENT_COST", "father_educ")
  enrolled_only_missing_vars <- c("TUTION_FEE", "EXAMINATION_FEE", "OTHER_FEES_PAYMENTS",
↪  "BOOKS", "STATIONERY", "UNIFORM", "TRANSPORT")

  # Legacy Chunk 8 distinguished variables used in the probit from
  # enrolled-only expenditure variables.  Keep that distinction so the total
  # row labelled "probit-model" is not inflated by fee variables that are
  # undefined for non-enrolled children.
  probit_vars <- c(
    "enrolled", "ENROLLED", "AGE", "age", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP",
    "SECTOR", "state_0708", "region_0708", all_child_missing_vars
  )
  probit_vars <- intersect(probit_vars, names(df))
  if (!length(probit_vars)) probit_vars <- setdiff(names(df),
  ↪  intersect(enrolled_only_missing_vars, names(df)))
```

```
  list(
    probit_vars = probit_vars,
    miss_vars_all = intersect(all_child_missing_vars, names(df)),
    miss_vars_enrolled = intersect(enrolled_only_missing_vars, names(df)),
    group_vars = intersect(c("SECTOR", "SEX", "RELIGION", "SOCIAL_GROUP", "state_0708"),
     ↪ names(df)),
    cts_vars = intersect(c("AGE", "HH_SIZE"), names(df)),
    regional_vars = intersect(c("state_0708", "region_0708"), names(df))
  )
}

#' diagnose missingness
#'
#' Port the legacy Chunk 8 missingness diagnostics into an opt-in diagnostic
#' object.  The returned list preserves the legacy sequence: variable counts,
#' regional rankings, missingness correlation matrices, BH-adjusted logit screens,
#' and notes for commented case-study / chi-square checks.
diagnose_missingness <- function(selection_data, cfg) {
  df <- as.data.frame(selection_data, stringsAsFactors = FALSE)
  vars <- missingness_variables(df)
  probit_vars <- vars$probit_vars

  counts <- summarize_missingness_by_variable(df[probit_vars])
  if (length(probit_vars)) {
    total_na <- sum(!stats::complete.cases(df[probit_vars]))
    total_complete <- sum(stats::complete.cases(df[probit_vars]))
  } else {
    total_na <- 0L
    total_complete <- nrow(df)
  }
  counts <- safe_bind_rows(list(
    counts,
    data.frame(missing_var = "Total probit-model with NA", n_missing = total_na, pct_missing
     ↪ = if (nrow(df)) total_na / nrow(df) else NA_real_),
    data.frame(missing_var = "Total probit-model with no NA", n_missing = total_complete,
     ↪ pct_missing = if (nrow(df)) total_complete / nrow(df) else NA_real_)
  ))

  regional <- summarize_missingness_regions(df, probit_vars, vars)
  corr_all <- missingness_correlation_matrix(df, vars$miss_vars_all, vars$group_vars,
↪  vars$cts_vars)
  enrolled_rows <- enrolled_schooling_rows(df)
  corr_enrolled <- missingness_correlation_matrix(df[enrolled_rows, , drop = FALSE],
↪  vars$miss_vars_enrolled, vars$group_vars, vars$cts_vars)

  covars <- intersect(c("SECTOR", "SEX", "AGE", "HH_SIZE", "RELIGION", "SOCIAL_GROUP",
↪  "state_0708"), names(df))
  miss_all <- intersect(vars$miss_vars_all, names(df))
  miss_enrolled <- intersect(vars$miss_vars_enrolled, names(df))
  logit_all <- if (length(miss_all) && length(covars)) run_missingness_logits(df, miss_all,
↪  covars) else data.frame()
  logit_enrolled <- if (length(miss_enrolled) && length(covars) && any(enrolled_rows))
↪  run_missingness_logits(df[enrolled_rows, , drop = FALSE], miss_enrolled, covars) else
↪  data.frame()
```

```r
  logit_summary <- summarize_missingness_logits(safe_bind_rows(list(logit_all,
↪  logit_enrolled)))
  case_study <- summarize_missingness_case_study(df, vars)
  chi_square <- summarize_missingness_chisq(df, probit_vars)

  notes <- data.frame(
    diagnostic = c(
      "rajasthan_southern_case_study",
      "chi_square_any_na_by_state",
      "chi_square_any_na_by_region",
      "parallel_missingness_logit"
    ),
    legacy_status = c(
      "commented diagnostic View() code preserved as documented note",
      "commented diagnostic test not run by default",
      "commented diagnostic test not run by default",
      "ported using lapply for reproducible targets execution; legacy mclapply/parLapply
       ↪  choice documented"
    ),
    stringsAsFactors = FALSE
  )

  out <- list(
    missing_counts = counts,
    regional_cost = regional$cost,
    regional_distance = regional$distance,
    regional_father_education = regional$father_education,
    corr_all = corr_all,
    corr_enrolled = corr_enrolled,
    logit_all = logit_all,
    logit_enrolled = logit_enrolled,
    logit_summary = logit_summary,
    case_study = case_study,
    chi_square = chi_square,
    notes = notes
  )
  class(out) <- c("emi_missingness_diagnostics", class(out))
  out
}

enrolled_schooling_rows <- function(df) {
  if (!"enrolled" %in% names(df)) return(rep(TRUE, nrow(df)))
  value <- tolower(as.character(df$enrolled))
  value %in% c("yes", "1", "true", "enrolled")
}

summarize_missingness_by_variable <- function(df) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!length(names(df))) {
    return(data.frame(missing_var = character(), n_missing = integer(), pct_missing =
     ↪  numeric(), stringsAsFactors = FALSE))
  }
  data.frame(
    missing_var = names(df),
    n_missing = vapply(df, function(x) sum(is.na(x)), integer(1)),
```

```r
    pct_missing = if (nrow(df)) vapply(df, function(x) mean(is.na(x)), numeric(1)) else NA_real_,
    stringsAsFactors = FALSE
  )
}


summarize_missingness_case_study <- function(df, vars) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!nrow(df)) return(data.frame())
  state_col <- intersect(c("state_0708", "state"), names(df))
  region_col <- intersect(c("region_0708", "region"), names(df))
  if (!length(state_col)) return(data.frame(note = "No state column available for Rajasthan/Southern case study.", stringsAsFactors = FALSE))

  keep <- grepl("rajasthan", as.character(df[[state_col[[1]]]]), ignore.case = TRUE)
  case_scope <- "Rajasthan"
  if (length(region_col)) {
    region_keep <- grepl("southern", as.character(df[[region_col[[1]]]]), ignore.case = TRUE)
    if (any(keep & region_keep, na.rm = TRUE)) {
      keep <- keep & region_keep
      case_scope <- "Rajasthan / Southern"
    }
  }
  vars_to_check <- intersect(c(vars$miss_vars_all, vars$miss_vars_enrolled), names(df))
  if (!length(vars_to_check) || !any(keep, na.rm = TRUE)) {
    return(data.frame(case_scope = case_scope, n_rows = sum(keep, na.rm = TRUE), note = "No matching rows or missingness variables available.", stringsAsFactors = FALSE))
  }
  case_df <- df[keep, vars_to_check, drop = FALSE]
  any_missing <- !stats::complete.cases(case_df)
  data.frame(
    case_scope = case_scope,
    n_rows = nrow(case_df),
    n_rows_with_any_missing = sum(any_missing),
    n_rows_with_partial_missing = sum(any_missing & rowSums(is.na(case_df)) < ncol(case_df)),
    n_missing_cells = sum(is.na(case_df)),
    n_observed_cells_in_rows_with_missing = sum(!is.na(case_df[any_missing, , drop = FALSE])),
    interpretation = "Current analog of the legacy Rajasthan/Southern View() check: rows with partial cost-variable missingness preserve the legacy concern that a child was not necessarily excluded wholesale when one cost field was missing.",
    stringsAsFactors = FALSE
  )
}

summarize_missingness_chisq <- function(df, probit_vars) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!nrow(df) || !length(probit_vars)) return(data.frame())
  probit_vars <- intersect(probit_vars, names(df))
  if (!length(probit_vars)) return(data.frame())
  any_na <- !stats::complete.cases(df[probit_vars])
  safe_test <- function(group_col, label) {
```

```r
    if (!group_col %in% names(df)) {
      return(data.frame(test = label, status = "not_available", reason = paste("Missing
       ↪  column", group_col), stringsAsFactors = FALSE))
    }
    group <- as.character(df[[group_col]])
    keep <- !is.na(group) & nzchar(group)
    if (length(unique(group[keep])) < 2L || length(unique(any_na[keep])) < 2L) {
      return(data.frame(test = label, status = "not_estimated", reason = "Insufficient
       ↪  variation for chi-square test.", stringsAsFactors = FALSE))
    }
    tab <- table(any_na = any_na[keep], group = group[keep])
    fit <- suppressWarnings(stats::chisq.test(tab))
    data.frame(
      test = label,
      status = "estimated",
      statistic = unname(fit$statistic),
      parameter = unname(fit$parameter),
      p.value = unname(fit$p.value),
      n = sum(tab),
      method = fit$method,
      stringsAsFactors = FALSE
    )
  }
  safe_bind_rows(list(
    safe_test("state_0708", "any_probit_model_na_by_state"),
    safe_test("region_0708", "any_probit_model_na_by_region")
  ))
}

summarize_missingness_regions <- function(df, probit_vars, vars, top_n = 20L) {
  empty <- data.frame()
  if (!"state_0708" %in% names(df)) {
    return(list(cost = empty, distance = empty, father_education = empty))
  }
  if (!length(probit_vars)) probit_vars <- names(df)
  temp <- df
  region_cols <- intersect(c("state_0708", "region_0708"), names(temp))
  # Legacy Chunk 8 inspected both state and region.  Some cleaned selection
  # inputs no longer expose region_0708, so keep the diagnostic alive at the
  # state level instead of writing empty CSVs.
  if (!"region_0708" %in% region_cols) {
    temp$region_0708 <- "all_regions_available_in_state_only_input"
    region_cols <- c("state_0708", "region_0708")
  }

  temp$any_na_row <- !stats::complete.cases(temp[intersect(probit_vars, names(temp))])
  for (nm in c("dmean_num_ENROLLMENT_COST", "DIST_FROM_NEAREST_PRIMARY_CLASS",
   ↪  "father_educ")) {
    temp[[paste0("miss_", nm)]] <- if (nm %in% names(temp)) as.integer(is.na(temp[[nm]]))
 ↪  else NA_integer_
  }
  temp$is_urban <- if ("SECTOR" %in% names(temp)) as.integer(temp$SECTOR == "Urban") else
 ↪  NA_integer_
  temp$is_female <- if ("SEX" %in% names(temp)) as.integer(temp$SEX == "Female") else
 ↪  NA_integer_
```

```r
  temp$is_hindu <- if ("RELIGION" %in% names(temp)) as.integer(temp$RELIGION == "Hindu")
↪ else NA_integer_
  temp$is_muslim <- if ("RELIGION" %in% names(temp)) as.integer(temp$RELIGION == "Muslim")
↪ else NA_integer_
  temp$is_st_sc_obc <- if ("SOCIAL_GROUP" %in% names(temp)) as.integer(temp$SOCIAL_GROUP
↪ %in% c("Scheduled Tribe", "Scheduled Caste", "Other Backward Class")) else NA_integer_

  measure_cols <- c("any_na_row", "miss_dmean_num_ENROLLMENT_COST",
↪ "miss_DIST_FROM_NEAREST_PRIMARY_CLASS", "miss_father_educ", "is_urban", "is_female",
↪ "is_hindu", "is_muslim", "is_st_sc_obc")
  grouped <- stats::aggregate(
    temp[measure_cols],
    temp[region_cols],
    function(x) mean(as.numeric(x), na.rm = TRUE)
  )
  n <- stats::aggregate(temp$any_na_row, temp[region_cols], length)
  names(n)[names(n) == "x"] <- "n"
  out <- merge(n, grouped, by = region_cols, all = TRUE)
  names(out) <- sub("^any_na_row$", "pct_any_na", names(out))
  out$region_diagnostic_level <- if ("region_0708" %in% names(df)) "state_region" else
↪ "state_only_fallback"
  pct_cols <- setdiff(names(out), c(region_cols, "n", "region_diagnostic_level"))
  out[pct_cols] <- lapply(out[pct_cols], function(x) round(100 * x, 2))
  rank_one <- function(col) {
    if (!col %in% names(out)) return(empty)
    out[order(-out[[col]]), , drop = FALSE][seq_len(min(top_n, nrow(out))), , drop = FALSE]
  }
  list(
    cost = rank_one("miss_dmean_num_ENROLLMENT_COST"),
    distance = rank_one("miss_DIST_FROM_NEAREST_PRIMARY_CLASS"),
    father_education = rank_one("miss_father_educ")
  )
}

missingness_correlation_matrix <- function(df, miss_vars, group_vars, cts_vars) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  miss_vars <- intersect(miss_vars, names(df))
  if (!length(miss_vars) || !nrow(df)) return(matrix(numeric(), nrow = 0L, ncol = 0L))
  miss_df <- as.data.frame(lapply(df[miss_vars], function(x) as.integer(is.na(x))),
↪ check.names = FALSE)
  names(miss_df) <- paste0("miss_", names(miss_df))
  cts <- intersect(cts_vars, names(df))
  cts_df <- if (length(cts)) as.data.frame(lapply(df[cts], numeric_like), check.names =
↪ FALSE) else data.frame()
  grp <- intersect(group_vars, names(df))
  grp_df <- if (length(grp)) {
    grp_data <- as.data.frame(lapply(df[grp], function(x) {
      x <- as.character(x)
      x[is.na(x) | !nzchar(x)] <- "missing"
      factor(x)
    }), check.names = FALSE)
    # Legacy Chunk 8 used model.matrix-style expansions of sector, sex,
    # religion, social group, state, and related grouping variables before
    # computing missingness correlations.  Small diagnostic subsets, especially
    # enrolled-only slices used in tests or regional filters, may contain only a
```

```r
    # single observed level for one or more grouping variables.  model.matrix()
    # cannot apply contrasts to one-level factors, so drop constant grouping
    # variables while preserving all grouping variables with real variation.
    varying <- vapply(grp_data, function(x) length(unique(as.character(x))) > 1L,
 ↪  logical(1))
    grp_data <- grp_data[varying]
    if (length(grp_data)) {
      stats::model.matrix(~ . - 1, data = grp_data)
    } else {
      matrix(numeric(), nrow = nrow(df), ncol = 0L)
    }
  } else matrix(numeric(), nrow = nrow(df), ncol = 0L)
  safe_pairwise_cor(cbind(miss_df, cts_df, grp_df))
}


binomial_fit_issues <- function(fit, warnings = character(), tolerance =
 ↪  sqrt(.Machine$double.eps)) {
  issues <- unique(as.character(warnings[nzchar(warnings)]))
  probabilities <- stats::fitted(fit)
  boundary_fit <- any(
    is.finite(probabilities) &
      (probabilities <= tolerance | probabilities >= 1 - tolerance)
  )
  if (boundary_fit) {
    issues <- c(issues, "fitted probabilities are numerically near 0 or 1")
  }
  if (!isTRUE(fit$converged)) {
    issues <- c(issues, "binomial model did not converge")
  }
  unique(issues)
}

#' check missing logit parallel
#'
#' Legacy Chunk 8 used mclapply/parLapply after defining the same one-logit-per-
#' missing-variable problem.  Targets already parallelizes at the target level,
#' so this implementation keeps deterministic in-process lapply while preserving
#' the model specification and BH adjustment.
check_missing_logit_parallel <- function(df, miss_vars, covars, method_p = "BH") {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  miss_vars <- intersect(miss_vars, names(df))
  covars <- intersect(covars, names(df))
  if (!length(miss_vars) || !length(covars)) return(data.frame())
  rhs <- paste(covars, collapse = " + ")
  fit_one <- function(m) {
    y <- is.na(df[[m]])
    if (length(unique(y)) < 2L) return(data.frame())
    f <- stats::as.formula(paste0("is.na(`", m, "`) ~ ", rhs))
    tryCatch({
      fit_warnings <- character()
      fit <- withCallingHandlers(
        stats::glm(f, data = df, family = stats::binomial),
        warning = function(w) {
          fit_warnings <<- c(fit_warnings, conditionMessage(w))
```

```r
          invokeRestart("muffleWarning")
        }
      )
      fit_issues <- binomial_fit_issues(fit, fit_warnings)
      pseudoR2 <- 1 - fit$deviance / fit$null.deviance
      out <- broom::tidy(fit)
      out$missing_var <- m
      out$pseudoR2 <- pseudoR2
      out$nobs <- stats::nobs(fit)
      out$status <- if (length(fit_issues)) "estimated_with_warning" else "estimated"
      out$reason <- if (length(fit_issues)) paste(fit_issues, collapse = "; ") else
 ↪  NA_character_
      out
    }, error = function(e) {
      data.frame(term = NA_character_, estimate = NA_real_, std.error = NA_real_, statistic
       ↪  = NA_real_, p.value = NA_real_, missing_var = m, pseudoR2 = NA_real_, nobs =
       ↪  length(y), status = "failed", reason = conditionMessage(e), stringsAsFactors =
       ↪  FALSE)
    })
  }
  out <- safe_bind_rows(lapply(miss_vars, fit_one))
  adjust_missingness_pvalues_bh(out, method_p = method_p)
}

run_missingness_logits <- function(df, miss_vars, covars) {
  check_missing_logit_parallel(df, miss_vars, covars)
}

adjust_missingness_pvalues_bh <- function(df, method_p = "BH") {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!nrow(df) || !"p.value" %in% names(df)) return(df)
  df$p_adj <- NA_real_
  idx <- !is.na(df$p.value) & df$term != "(Intercept)"
  df$p_adj[idx] <- stats::p.adjust(df$p.value[idx], method = method_p)
  df
}

summarize_missingness_logits <- function(df) {
  df <- as.data.frame(df, stringsAsFactors = FALSE)
  if (!nrow(df) || !all(c("missing_var", "p_adj", "pseudoR2") %in% names(df))) {
    return(data.frame(missing_var = character(), n_sig = integer(), pseudoR2 = numeric(),
     ↪  stringsAsFactors = FALSE))
  }
  split_df <- split(df, df$missing_var)
  out <- safe_bind_rows(lapply(split_df, function(x) {
    pmax <- suppressWarnings(max(x$pseudoR2, na.rm = TRUE))
    if (!is.finite(pmax)) pmax <- NA_real_
    data.frame(
      missing_var = x$missing_var[[1]],
      n_sig = sum(x$p_adj < 0.05 & x$term != "(Intercept)", na.rm = TRUE),
      pseudoR2 = pmax,
      stringsAsFactors = FALSE
    )
  }))
  out[order(-out$pseudoR2), , drop = FALSE]
```

```r
}

missingness_correlation_pairs <- function(mat, top_n = 50L) {
  mat <- as.matrix(mat)
  if (!length(mat) || nrow(mat) < 2L || ncol(mat) < 2L) {
    return(data.frame(var1 = character(), var2 = character(), correlation = numeric(),
     ↪  abs_correlation = numeric(), stringsAsFactors = FALSE))
  }
  mat[lower.tri(mat, diag = TRUE)] <- NA_real_
  idx <- which(is.finite(mat), arr.ind = TRUE)
  if (!nrow(idx)) {
    return(data.frame(var1 = character(), var2 = character(), correlation = numeric(),
     ↪  abs_correlation = numeric(), stringsAsFactors = FALSE))
  }
  out <- data.frame(
    var1 = rownames(mat)[idx[, "row"]],
    var2 = colnames(mat)[idx[, "col"]],
    correlation = mat[idx],
    stringsAsFactors = FALSE
  )
  out$abs_correlation <- abs(out$correlation)
  out <- out[order(-out$abs_correlation), , drop = FALSE]
  utils::head(out, top_n)
}

save_missingness_correlation_heatmap <- function(mat, path, max_vars = 35L, title =
 ↪  "Missingness correlation matrix") {
  mat <- as.matrix(mat)
  dir.create(dirname(path), recursive = TRUE, showWarnings = FALSE)
  if (!length(mat) || nrow(mat) < 2L || ncol(mat) < 2L) {
    grDevices::png(path, width = 900, height = 500, res = 120)
    graphics::plot.new()
    graphics::text(0.5, 0.5, "No correlation matrix available")
    grDevices::dev.off()
    return(normalizePath(path, mustWork = FALSE))
  }
  score <- apply(abs(mat), 1L, function(x) max(x[is.finite(x) & x < 1], na.rm = TRUE))
  score[!is.finite(score)] <- 0
  keep <- names(sort(score, decreasing = TRUE))[seq_len(min(max_vars, length(score)))]
  mat <- mat[keep, keep, drop = FALSE]
  grDevices::png(path, width = 1400, height = 1200, res = 140)
  old <- graphics::par(no.readonly = TRUE)
  on.exit({ graphics::par(old); grDevices::dev.off() }, add = TRUE)
  graphics::par(mar = c(11, 11, 4, 2))
  graphics::image(
    x = seq_len(ncol(mat)),
    y = seq_len(nrow(mat)),
    z = t(mat[nrow(mat):1, , drop = FALSE]),
    axes = FALSE,
    xlab = "",
    ylab = "",
    main = title,
    zlim = c(-1, 1)
  )
```

```r
  graphics::axis(1, at = seq_len(ncol(mat)), labels = colnames(mat), las = 2, cex.axis =
   ↪  0.55)
  graphics::axis(2, at = seq_len(nrow(mat)), labels = rev(rownames(mat)), las = 2, cex.axis
   ↪  = 0.55)
  graphics::box()
  normalizePath(path, mustWork = FALSE)
}


save_missingness_logit_plot <- function(summary_df, path, title = "Structured missingness
 ↪  logit screen", top_n = 25L) {
  dir.create(dirname(path), recursive = TRUE, showWarnings = FALSE)
  df <- as.data.frame(summary_df, stringsAsFactors = FALSE)
  grDevices::png(path, width = 1100, height = 850, res = 130)
  old <- graphics::par(no.readonly = TRUE)
  on.exit({ graphics::par(old); grDevices::dev.off() }, add = TRUE)
  if (!nrow(df) || !all(c("missing_var", "pseudoR2") %in% names(df))) {
    graphics::plot.new()
    graphics::text(0.5, 0.5, "No missingness-logit summary available")
    return(normalizePath(path, mustWork = FALSE))
  }
  df <- df[is.finite(df$pseudoR2), , drop = FALSE]
  if (!nrow(df)) {
    graphics::plot.new()
    graphics::text(0.5, 0.5, "No finite pseudo-R² values available")
    return(normalizePath(path, mustWork = FALSE))
  }
  df <- df[order(df$pseudoR2, decreasing = TRUE), , drop = FALSE]
  df <- utils::head(df, top_n)
  graphics::par(mar = c(5, 13, 4, 2))
  graphics::barplot(
    rev(df$pseudoR2),
    names.arg = rev(df$missing_var),
    horiz = TRUE,
    las = 1,
    xlab = "Pseudo-R² (predictability of missingness)",
    main = title
  )
  normalizePath(path, mustWork = FALSE)
}

save_missingness_diagnostics <- function(diagnostics, dir =
 ↪  "outputs/diagnostics/extended/missingness") {
  dir.create(dir, recursive = TRUE, showWarnings = FALSE)
  if (!inherits(diagnostics, "emi_missingness_diagnostics")) diagnostics <-
   ↪  list(missing_counts = as.data.frame(diagnostics))
  paths <- c(
    missing_counts = write_diagnostic_csv(diagnostics$missing_counts %||% data.frame(),
     ↪  file.path(dir, "missingness_counts.csv")),
    regional_cost = write_diagnostic_csv(diagnostics$regional_cost %||% data.frame(),
     ↪  file.path(dir, "regional_missingness_cost.csv")),
    regional_distance = write_diagnostic_csv(diagnostics$regional_distance %||%
     ↪  data.frame(), file.path(dir, "regional_missingness_distance.csv")),
    regional_father = write_diagnostic_csv(diagnostics$regional_father_education %||%
     ↪  data.frame(), file.path(dir, "regional_missingness_father_education.csv")),
```

```r
    logit_all = write_diagnostic_csv(diagnostics$logit_all %||% data.frame(), file.path(dir, "missingness_logits_all.csv")),
    logit_enrolled = write_diagnostic_csv(diagnostics$logit_enrolled %||% data.frame(), file.path(dir, "missingness_logits_enrolled.csv")),
    logit_summary = write_diagnostic_csv(diagnostics$logit_summary %||% data.frame(), file.path(dir, "missingness_logit_summary.csv")),
    case_study = write_diagnostic_csv(diagnostics$case_study %||% data.frame(), file.path(dir, "missingness_rajasthan_southern_case_study.csv")),
    chi_square = write_diagnostic_csv(diagnostics$chi_square %||% data.frame(), file.path(dir, "missingness_chi_square_tests.csv")),
    notes = write_diagnostic_csv(diagnostics$notes %||% data.frame(), file.path(dir, "missingness_legacy_notes.csv"))
  )
  if (length(diagnostics$corr_all)) {
    paths <- c(
      paths,
      corr_all = write_diagnostic_matrix(diagnostics$corr_all, file.path(dir, "missingness_correlation_all.csv")),
      corr_all_pairs = write_diagnostic_csv(missingness_correlation_pairs(diagnostics$corr_all), file.path(dir, "missingness_correlation_all_top_pairs.csv")),
      corr_all_heatmap = save_missingness_correlation_heatmap(diagnostics$corr_all, file.path(dir, "missingness_correlation_all.png"), title = "Missingness correlations: all observations")
    )
  }
  if (length(diagnostics$corr_enrolled)) {
    paths <- c(
      paths,
      corr_enrolled = write_diagnostic_matrix(diagnostics$corr_enrolled, file.path(dir, "missingness_correlation_enrolled.csv")),
      corr_enrolled_pairs = write_diagnostic_csv(missingness_correlation_pairs(diagnostics$corr_enrolled), file.path(dir, "missingness_correlation_enrolled_top_pairs.csv")),
      corr_enrolled_heatmap = save_missingness_correlation_heatmap(diagnostics$corr_enrolled, file.path(dir, "missingness_correlation_enrolled.png"), title = "Missingness correlations: enrolled observations")
    )
  }
  if (nrow(diagnostics$logit_summary %||% data.frame())) {
    paths <- c(
      paths,
      logit_pseudo_r2_plot = save_missingness_logit_plot(
        diagnostics$logit_summary,
        file.path(dir, "missingness_logit_pseudo_r2.png"),
        title = "How structured is missingness?"
      )
    )
  }
  output_manifest(paths)
}
```

## Survey-weighted enrollment probit

Source: R/selection/estimate_selection_probit.R

```r
selection_probit_variables <- function(selection_data) {
  controls <- c("AGE", "age", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR")
  exclusion_all_kids <- c("DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ")
  exclusion_district <- c(
    "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
    "dmean_num_RECD_SCHOLARSHIP_STIPEND", "dmean_num_RECD_TXT_BOOKS",
    "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_RECD",
    "dmean_num_ENROLLMENT_COST"
  )
  intersect(c(controls, exclusion_all_kids, exclusion_district), names(selection_data))
}

#' estimate selection probit
#'
estimate_selection_probit <- function(selection_data, cfg) {
  if (!"enrolled" %in% names(selection_data) || all(is.na(selection_data$enrolled))) {
    return(list(status = "out_of_active_pipeline", reason = "No enrolled variable."))
  }
  covars <- selection_probit_variables(selection_data)
  if (!length(covars)) {
    return(list(status = "out_of_active_pipeline", reason = "No probit covariates."))
  }
  f_probit <- stats::reformulate(covars, response = "enrolled")
  if (identical(cfg$mode, "final") && requireNamespace("survey", quietly = TRUE)) {
    design <- build_survey_design_selection(selection_data)
    if (!is.null(design)) {
      out <- fit_selection_probit(design, f_probit)
      return(stabilize_selection_model_formula(out, f_probit))
    }
  }
  out <- stats::glm(f_probit, data = selection_data, family = stats::binomial(link =
  ↪  "probit"))
  stabilize_selection_model_formula(out, f_probit)
}

#' store a durable selection-model formula
#'
#' Programmatically fitted models otherwise retain a call to the local symbol
#' `f_probit`. Packages which reconstruct model data from the saved call cannot
#' resolve that symbol after the model is serialized by targets. Embed the
#' formula object in the call and retain the existing audit attribute.
stabilize_selection_model_formula <- function(model, formula) {
  if (!inherits(formula, "formula")) {
    stop("Selection model formula must inherit from formula.", call. = FALSE)
  }
  if (!is.null(model$call)) model$call$formula <- formula
  attr(model, "selection_probit_formula") <- formula
  model
}

#' build survey design selection
#'
```

```r
build_survey_design_selection <- function(selection_df) {
  psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
  weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
  strata_cols <- intersect(c("STATE", "STRATUM", "SUB_STRATUM_NO"), names(selection_df))
  if (is.null(psu) || is.null(weight) || !length(strata_cols)) return(NULL)
  options(survey.lonely.psu = "average")
  selection_df$.survey_strata <- interaction(selection_df[strata_cols], drop = TRUE)
  survey::svydesign(
    ids = stats::as.formula(paste0("~", psu)),
    strata = ~.survey_strata,
    weights = stats::as.formula(paste0("~", weight)),
    data = selection_df,
    nest = TRUE
  )
}

#' fit selection probit
#'
fit_selection_probit <- function(selection_design, f_probit) {
  survey::svyglm(f_probit, design = selection_design, family = stats::quasibinomial(link =
  ↪  "probit"))
}
```

### Average marginal effects via automatic differentiation

Source: `R/selection/compute_average_marginal_effects.R`

```r
# Use automatic differentiation for SE delta-method gradients, as in the legacy Rmd.

#' compute average marginal effects
#'
compute_average_marginal_effects <- function(selection_model, selection_data, cfg = list())
  ↪  {
  if (!inherits(selection_model, "glm")) {
    return(ame_out_of_pipeline(
      selection_model$status %||% "out_of_active_pipeline",
      selection_model$reason %||% "Selection model not estimated in smoke mode"
    ))
  }

  # `selection_model` is often a survey::svyglm object loaded from the targets
  # store. Survey lonely-PSU handling is an R option, not a durable model
  # attribute, so it may not be set in the downstream AME target even though it
  # was set when the model was estimated. Set it explicitly around all AME
  # computations to make reruns deterministic and warning/error-free.
  old_lonely_psu <- getOption("survey.lonely.psu")
  old_domain_lonely <- getOption("survey.adjust.domain.lonely")
  options(survey.lonely.psu = "average", survey.adjust.domain.lonely = TRUE)
  on.exit({
    if (is.null(old_lonely_psu)) {
      options(survey.lonely.psu = NULL)
    } else {
      options(survey.lonely.psu = old_lonely_psu)
    }
    if (is.null(old_domain_lonely)) {
      options(survey.adjust.domain.lonely = NULL)
```

```
    } else {
      options(survey.adjust.domain.lonely = old_domain_lonely)
    }
  }, add = TRUE)

#' run expression while muffling survey lonely-PSU warnings
#'
#' survey emits lonely-PSU warnings even when survey.lonely.psu is set to
#' average and the adjustment is intentional. Muffle only those handled
#' warnings inside the AME target so strict final builds remain warning-clean.
muffle_survey_lonely_psu_warnings <- function(expr) {
  withCallingHandlers(
    expr,
    warning = function(w) {
      msg <- conditionMessage(w)
      lonely_psu_warning <- grepl(
        "has only one PSU at stage",
        msg,
        fixed = TRUE
      )
      if (lonely_psu_warning) invokeRestart("muffleWarning")
    }
  )
}


  if (!isTRUE(cfg$run_full_ame)) {
    return(format_ame_results(data.frame(
      term = names(stats::coef(selection_model)),
      estimate = unname(stats::coef(selection_model)),
      std.error = NA_real_,
      statistic = NA_real_,
      p.value = NA_real_,
      s.value = NA_real_,
      conf.low = NA_real_,
      conf.high = NA_real_,
      method = "coefficient_fallback",
      status = "estimated",
      reason = "Draft config uses coefficient fallback instead of full AME computation"
    )))
  }

  if (!requireNamespace("marginaleffects", quietly = TRUE)) {
    return(ame_out_of_pipeline(
      "out_of_active_pipeline",
      "Package marginaleffects is required for full AME computation"
    ))
  }

  out <- muffle_survey_lonely_psu_warnings(
    tryCatch(
      compute_ames_autodiff(selection_model, selection_data),
      error = function(e) {
        fallback <- compute_ames_probit_analytic(selection_model, selection_data)
        fallback$method <- "delta_method_analytic_probit"
```

```r
        fallback$reason <- NA_character_
        fallback
      }
    )
  )
  format_ame_results(out)
}

ame_out_of_pipeline <- function(status, reason) {
  data.frame(
    term = NA_character_,
    estimate = NA_real_,
    std.error = NA_real_,
    statistic = NA_real_,
    p.value = NA_real_,
    s.value = NA_real_,
    conf.low = NA_real_,
    conf.high = NA_real_,
    method = "not_run",
    status = status,
    reason = reason
  )
}

#' extract model-frame data and AME weights
#'
#' marginaleffects warns, correctly, when weighted/survey models are summarized
#' without an explicit `wts` argument. Build a newdata frame from the exact
#' model estimation sample and attach a synthetic weight column whenever the
#' fitted model exposes usable weights.
ame_model_weight <- function(model_data, model_weights = NULL) {
  weight_cols <- intersect(c("weight", "WEIGHT", "Multiplier", "multiplier", "(weights)"),
  ↪  names(model_data))
  if (length(weight_cols)) {
    return(suppressWarnings(as.numeric(model_data[[weight_cols[[1]]]])))
  }

  if (!is.null(model_weights) && length(model_weights) == nrow(model_data)) {
    return(suppressWarnings(as.numeric(model_weights)))
  }

  NULL
}

#' build marginaleffects newdata and weights
#'
#' @return List with `data`, `wts`, and `has_explicit_weights`.
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))
  model_weights <- tryCatch(stats::weights(model), error = function(e) NULL)
  weight <- ame_model_weight(model_data, model_weights)

  if (!is.null(weight) && length(weight) == nrow(model_data) && any(is.finite(weight) &
   ↪  weight != 1)) {
    model_data$.ame_weight <- weight
```

```r
    return(list(data = model_data, wts = ".ame_weight", has_explicit_weights = TRUE))
  }

  list(data = model_data, wts = FALSE, has_explicit_weights = FALSE)
}

#' run marginaleffects while muffling only a redundant explicit-weight warning
#'
run_avg_slopes <- function(model, model_data, wts, numderiv = NULL) {
  args <- list(
    model = model,
    newdata = model_data,
    wts = wts,
    vcov = TRUE,
    type = "response"
  )
  if (!is.null(numderiv)) args$numderiv <- numderiv

  withCallingHandlers(
    do.call(marginaleffects::avg_slopes, args),
    warning = function(w) {
      msg <- conditionMessage(w)
      explicit_weight_warning <- grepl(
        "normally good practice to specify weights using the `wts` argument",
        msg,
        fixed = TRUE
      )
      if (explicit_weight_warning && !identical(wts, FALSE)) {
        invokeRestart("muffleWarning")
      }
    }
  )
}

#' compute ames autodiff
#'
compute_ames_autodiff <- function(model, newdata) {
  # Use the exact estimation sample retained by glm/svyglm. Passing the full
  # pre-model selection_data can have extra rows omitted during model fitting,
  # which makes marginaleffects recycle weights/covariates silently.
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(model, amed$data, amed$wts)
}


#' compute analytic probit AMEs
#'
#' @return Data frame of observed-data average marginal effects.
compute_ames_probit_analytic <- function(model, newdata) {
  coefs <- stats::coef(model)
  coef_table <- as.data.frame(summary(model)$coefficients)
  terms <- setdiff(names(coefs), "(Intercept)")
  if (!length(terms)) return(ame_out_of_pipeline("out_of_active_pipeline", "Selection model
   ↪ has no non-intercept terms."))
```

```r
  newdata <- stats::model.frame(model)
  weights <- if ("weight" %in% names(newdata)) num(newdata$weight) else rep(1, nrow(newdata))
  eta <- stats::predict(model, type = "link")
  mean_phi <- wmean(stats::dnorm(eta), weights)
  factor_levels <- model$xlevels %||% list()

  rows <- lapply(terms, function(term) {
    factor_var <- names(factor_levels)[vapply(names(factor_levels), function(v) startsWith(term, v), logical(1))]
    estimate <- NA_real_
    if (length(factor_var)) {
      var <- factor_var[[which.max(nchar(factor_var))]]
      level <- sub(paste0("^", var), "", term)
      if (level %in% factor_levels[[var]]) {
        hi <- newdata
        lo <- newdata
        hi[[var]] <- factor(level, levels = factor_levels[[var]])
        lo[[var]] <- factor(factor_levels[[var]][[1]], levels = factor_levels[[var]])
        estimate <- wmean(stats::predict(model, newdata = hi, type = "response") - stats::predict(model, newdata = lo, type = "response"), weights)
      }
    } else {
      estimate <- unname(coefs[[term]]) * mean_phi
    }
    p_col <- intersect(c("Pr(>|z|)", "Pr(>|t|)"), names(coef_table))
    p <- if (term %in% rownames(coef_table) && length(p_col)) coef_table[term, p_col[[1]]] else NA_real_
    se_col <- intersect(c("Std. Error", "std.error"), names(coef_table))
    se_beta <- if (term %in% rownames(coef_table) && length(se_col)) coef_table[term, se_col[[1]]] else NA_real_
    se <- if (is.finite(se_beta)) abs(se_beta * mean_phi) else NA_real_
    z <- if (is.finite(se) && se > 0) estimate / se else NA_real_
    p_out <- if (is.finite(z)) 2 * stats::pnorm(abs(z), lower.tail = FALSE) else p
    data.frame(
      term = term,
      estimate = estimate,
      std.error = se,
      statistic = z,
      p.value = p_out,
      s.value = if (is.finite(p_out) && p_out > 0) -log2(p_out) else NA_real_,
      conf.low = if (is.finite(se)) estimate - 1.96 * se else NA_real_,
      conf.high = if (is.finite(se)) estimate + 1.96 * se else NA_real_,
      method = "delta_method_analytic_probit",
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  })
  do.call(rbind, rows)
}


is_marginaleffects_object <- function(x) {
```

```r
  inherits(x, c("marginaleffects", "comparisons", "avg_comparisons", "slopes", "avg_slopes",
   ↪  "predictions"))
}

copy_first_ame_column <- function(out, target, candidates) {
  if (!target %in% names(out)) out[[target]] <- NA
  for (candidate in candidates) {
    if (!candidate %in% names(out)) next
    missing_target <- is.na(out[[target]])
    if (any(missing_target)) out[[target]][missing_target] <-
     ↪  out[[candidate]][missing_target]
  }
  out
}

normalize_ame_columns <- function(out) {
  # marginaleffects has used both dotted and snake_case names across versions
  # and model classes. Normalize here before selecting the public/audit schema
  # so uncertainty columns are not silently dropped downstream.
  aliases <- list(
    std.error = c("std_error", "std.err", "Std. Error"),
    statistic = c("z", "z.value", "z_value", "t", "t.value", "t_value"),
    p.value = c("p_value", "p", "Pr(>|z|)", "Pr(>|t|)"),
    s.value = c("s_value"),
    conf.low = c("conf_low", "conf.low", "2.5 %"),
    conf.high = c("conf_high", "conf.high", "97.5 %")
  )
  for (target in names(aliases)) {
    out <- copy_first_ame_column(out, target, aliases[[target]])
  }
  out
}

ame_label_lookup <- function() {
  data.frame(
    term = c(
      "AGE", "SEX", "HH_SIZE",
      "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION",
      "SOCIAL_GROUP", "SOCIAL_GROUP", "SOCIAL_GROUP",
      "SECTOR",
      "DIST_FROM_NEAREST_PRIMARY_CLASS", "DIST_FROM_NEAREST_PRIMARY_CLASS",
       ↪  "DIST_FROM_NEAREST_PRIMARY_CLASS", "DIST_FROM_NEAREST_PRIMARY_CLASS",
      "father_educ", "father_educ", "father_educ", "father_educ", "father_educ",
       ↪  "father_educ", "father_educ",
      "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
       ↪  "dmean_num_RECD_SCHOLARSHIP_STIPEND",
      "dmean_num_RECD_TXT_BOOKS", "dmean_num_RECD_STATIONERY",
       ↪  "dmean_num_MID_DAY_MEAL_ETC_RECD", "dmean_num_ENROLLMENT_COST"
    ),
    contrast = c(
      "dY/dX", "Female - Male", "dY/dX",
      "Muslim - Hindu", "Christian - Hindu", "Sikh - Hindu", "Jain - Hindu", "Buddhist -
       ↪  Hindu", "Zoroastrian - Hindu", "Other - Hindu",
      "Scheduled Tribe - Other", "Scheduled Caste - Other", "Other Backward Class - Other",
      "Urban - Rural",
```

```r
      "1km <= d <2kms - d<1km", "2kms<= d <3kms - d<1km", "3kms <= d <5kms - d<1km",
       ↪ "d>=5kms - d<1km",
      "Literate, no school - Illiterate", "Literate, school < primary - Illiterate",
       ↪ "Primary - Illiterate", "Upper primary - Illiterate", "Secondary - Illiterate",
       ↪ "Higher secondary - Illiterate", "Postsecondary+ - Illiterate",
      "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX"
    ),
    Term = c(
      "Age (years)", "Female (ref: Male)", "Household size",
      "Religion: Muslim (ref: Hindu)", "Religion: Christian", "Religion: Sikh", "Religion:
       ↪ Jain", "Religion: Buddhist", "Religion: Zoroastrian", "Religion: Other",
      "Social group: Scheduled Tribe (ref: Other)", "Social group: Scheduled Caste", "Social
       ↪ group: Other Backward Class",
      "Urban (ref: Rural)",
      "Distance 1-2km (ref: <1km)", "Distance 2-3km", "Distance 3-5km", "Distance > 5km",
      "Father's educ.: Literate, no school (ref: Illiterate)", "Father's educ.: Literate,
       ↪ school < primary", "Father's educ.: Primary", "Father's educ.: Upper primary",
       ↪ "Father's educ.: Secondary", "Father's educ.: Higher secondary", "Father's educ.:
       ↪ Postsecondary+",
      "Educ. free available (ref: No)", "Tuition waiver received", "Scholarship/Stipend
       ↪ received", "Textbook(s) received", "Stationery received", "Mid-day meal, etc.
       ↪ received", "Enrollment cost (Rs.)"
    ),
    stringsAsFactors = FALSE
  )
}

ame_label_order <- function() ame_label_lookup()$Term

ame_modelsummary_label <- function(x) {
  x <- as.character(x)
  x <- gsub("\u00d7", ":", x, fixed = TRUE)
  x <- gsub("\\s+:\\s+", ": ", x, perl = TRUE)
  x <- gsub("\\(ref:\\s*", "(ref: ", x, perl = TRUE)
  x
}

infer_ame_contrast <- function(out) {
  if ("contrast" %in% names(out)) return(as.character(out$contrast))
  contrast <- rep("dY/dX", nrow(out))
  if (!"term" %in% names(out)) return(contrast)
  if ("comparison" %in% names(out)) contrast <- as.character(out$comparison)
  contrast[is.na(contrast) | !nzchar(contrast)] <- "dY/dX"
  contrast
}

ame_factor_base_terms <- function() {
  unique(ame_label_lookup()$term[ame_label_lookup()$contrast != "dY/dX"])
}

normalize_encoded_factor_ame_terms <- function(out) {
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  if (!"term" %in% names(out)) return(out)
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(out)
```

```r
  lookup <- ame_label_lookup()
  factor_terms <- ame_factor_base_terms()
  factor_terms <- factor_terms[order(nchar(factor_terms), decreasing = TRUE)]

  for (i in seq_len(nrow(out))) {
    current_term <- as.character(out$term[[i]])
    current_contrast <- as.character(out$contrast[[i]])
    if (!nzchar(current_term) || (!is.na(current_contrast) && current_contrast != "dY/dX"))
     ↪  next

    direct <- lookup$term == current_term & lookup$contrast == current_contrast
    if (any(direct)) next

    for (base_term in factor_terms) {
      if (!startsWith(current_term, base_term)) next
      level <- trimws(sub(paste0("^", base_term), "", current_term))
      if (!nzchar(level)) next
      candidates <- lookup[lookup$term == base_term, , drop = FALSE]
      contrast_hit <- candidates$contrast[startsWith(candidates$contrast, paste0(level, " -
 ↪  "))]
      if (length(contrast_hit)) {
        out$term[[i]] <- base_term
        out$contrast[[i]] <- contrast_hit[[1]]
        break
      }
    }
  }
  out
}

attach_ame_labels <- function(out) {
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  if (!"term" %in% names(out) && "variable" %in% names(out)) out$term <- out$variable
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(out)
  out <- normalize_encoded_factor_ame_terms(out)
  lookup <- ame_label_lookup()
  labeled <- merge(out, lookup, by = c("term", "contrast"), all.x = TRUE, sort = FALSE)
  labeled$Term[is.na(labeled$Term)] <- labeled$term[is.na(labeled$Term)]
  labeled$Term <- factor(labeled$Term, levels = unique(c(ame_label_order(), labeled$Term)))
  labeled <- labeled[order(labeled$Term), , drop = FALSE]
  labeled$Term <- as.character(labeled$Term)
  labeled
}


modelsummary_marginaleffects_object <- function(out, native_ame) {
  if (!is_marginaleffects_object(native_ame)) return(NULL)
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  required <- c("Term", "estimate", "std.error", "statistic", "p.value", "conf.low",
 ↪  "conf.high")
  if (!all(required %in% names(out))) return(native_ame)

  ms <- out[, required, drop = FALSE]
  ms$term <- ame_modelsummary_label(ms$Term)
  ms$contrast <- ""
```

```r
  ms$Term <- NULL
  # Preserve the marginaleffects class and non-structural attributes so
  # modelsummary can use its native marginaleffects/tidy pathway.  We still
  # hand modelsummary the labeled, ordered rows that are validated for the
  # public table, because passing the raw object caused modelsummary to reorder
  # contrasts and display labels against the wrong estimates.
  native_attributes <- attributes(native_ame)
  structural_attributes <- c("names", "row.names", "class")
  for (nm in setdiff(names(native_attributes), structural_attributes)) {
    attr(ms, nm) <- native_attributes[[nm]]
  }
  class(ms) <- unique(c(class(native_ame), class(ms)))
  ms
}

#' format ame results
#'
format_ame_results <- function(ame_results) {
  native_ame <- ame_results
  out <- tibble::as_tibble(ame_results)
  if (!"term" %in% names(out) && "variable" %in% names(out)) out$term <- out$variable
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(as.data.frame(out))
  out <- normalize_ame_columns(out)
  if (!"method" %in% names(out)) out$method <- "autodiff"
  if (!"status" %in% names(out)) out$status <- "estimated"
  if (!"reason" %in% names(out)) out$reason <- NA_character_
  labeled_terms <- unique(ame_label_lookup()$term)
  use_labeled_schema <- any(out$term %in% labeled_terms) ||
    any(grepl("^dmean_num_", out$term %||% character(), perl = TRUE)) ||
    any((out$contrast %||% "dY/dX") != "dY/dX", na.rm = TRUE)

  if (isTRUE(use_labeled_schema)) {
    out <- attach_ame_labels(out)
    required <- c(
      "Term", "term", "contrast", "estimate", "std.error", "statistic", "p.value",
       ↪  "s.value",
      "conf.low", "conf.high", "method", "status", "reason"
    )
  } else {
    required <- c(
      "term", "estimate", "std.error", "statistic", "p.value",
      "s.value", "conf.low", "conf.high", "method", "status", "reason"
    )
  }

  for (nm in setdiff(required, names(out))) out[[nm]] <- NA
  if ("p.value" %in% names(out) && "s.value" %in% names(out)) {
    missing_s <- is.na(out$s.value) & is.finite(out$p.value) & out$p.value > 0
    out$s.value[missing_s] <- -log2(out$p.value[missing_s])
  }
  out <- out[, required, drop = FALSE]
  if (is_marginaleffects_object(native_ame)) {
    attr(out, "marginaleffects_object") <- modelsummary_marginaleffects_object(out,
     ↪  native_ame)
  }
```

```r
  out
}
```

### District harmonization map and many-to-many QA

Source: `R/districts/build_district_join_map.R`

```r
#' Required reviewed district-crosswalk columns
#'
#' @return The name columns needed to attach 2001, 2007-08, 2017-18, and 2020
#'   source data to one harmonized district row.
district_join_map_required_columns <- function() {
  c("state_01", "district_01", "state_07", "district_07", "state_17", "district_17",
  ↪  "state_20", "district_20")
}

#' Prepare the reviewed district join map
#'
#' The active pipeline does not infer the harmonization map at build time. It
#' consumes the reviewed metadata crosswalk, gives each row a stable internal
#' identifier, and exposes empty diagnostic attributes until source attachment
#' produces a row-level match ledger in the planned district-matching rewrite.
prepare_district_join_map <- function(crosswalk) {
  out <- safe_df(crosswalk)
  missing <- setdiff(district_join_map_required_columns(), names(out))
  if (length(missing)) {
    stop(
      "District harmonization crosswalk is missing required columns: ",
      paste(missing, collapse = ", "),
      call. = FALSE
    )
  }
  if (!nrow(out)) stop("District harmonization crosswalk contains no rows.", call. = FALSE)

  out$.tracker_row <- seq_len(nrow(out))
  out$source <- "harmonization_crosswalk"
  out$match_status <- "reviewed_crosswalk_row"
  out$possible_false_positive <- FALSE
  out$many_to_many <- FALSE
  attr(out, "unmatched_rows") <- out[0, , drop = FALSE]
  attr(out, "possible_false_positives") <- out[0, , drop = FALSE]
  attr(out, "many_to_many_cases") <- out[0, , drop = FALSE]
  out
}

#' Flag many-to-many matches in a row-level match ledger
#'
#' This helper is retained for matching diagnostics. It is not used to infer the
#' reviewed crosswalk itself.
flag_many_to_many_matches <- function(join_map, source_col = NULL, tracker_col = NULL) {
  join_map <- safe_df(join_map)
  if (!nrow(join_map)) {
    join_map$many_to_many <- logical()
    join_map$many_to_many_type <- character()
    return(join_map)
  }
```

```
  source_col <- source_col %||% first_col(join_map, c(".source_row", "source_row", "source_key", "source_id"))
  tracker_col <- tracker_col %||% first_col(join_map, c(".tracker_row", "tracker_row", "tracker_key", "district_panel_id"))
  if (is.null(source_col) || is.null(tracker_col)) {
    join_map$many_to_many <- FALSE
    join_map$many_to_many_type <- "not_evaluable_missing_keys"
    return(join_map)
  }
  source_key <- canon(join_map[[source_col]])
  tracker_key <- canon(join_map[[tracker_col]])
  source_n <- ave(rep(1L, nrow(join_map)), source_key, FUN = length)
  tracker_n <- ave(rep(1L, nrow(join_map)), tracker_key, FUN = length)
  source_dup <- source_n > 1L & !is.na(source_key) & nzchar(source_key)
  tracker_dup <- tracker_n > 1L & !is.na(tracker_key) & nzchar(tracker_key)
  join_map$n_source_matches <- as.integer(source_n)
  join_map$n_tracker_matches <- as.integer(tracker_n)
  join_map$many_to_many <- source_dup & tracker_dup
  join_map$many_to_many_type <- ifelse(
    source_dup & tracker_dup, "many_to_many",
    ifelse(source_dup, "one_source_to_many_tracker",
      ifelse(tracker_dup, "many_source_to_one_tracker", "one_to_one"))
  )
  join_map
}
```

### Contiguity-based spatial weights

Source: `R/diagnostics/diagnose_spatial_weights.R`

```
#' build spatial weights
#'
#' Build the rook-contiguity spatial weights used by the legacy Rmd.
#'
#' The legacy chunk first removed rows with missing analysis values, then called
#' `poly2nb(queen = FALSE)`, `nb2mat(style = "B")`, and `nb2listw(style = "W")`.
#' The target stores all three objects so downstream diagnostics can reproduce
#' both the binary adjacency matrix and the row-standardized listw object.
#'
#' @return A list containing neighbor, matrix, and listw objects, or an explicit
#' inactive status when geometry is unavailable.
build_spatial_weights <- function(district_panel, cfg) {
  if (!inherits(district_panel, "sf")) {
    return(list(status = "out_of_active_pipeline", reason = "Requires sf geometry."))
  }
  need_pkg("spdep", "spatial weights")
  need_pkg("sf", "spatial weights")

  row_index <- spatial_weight_final_panel_rows(district_panel)
  coverage <- length(row_index) / max(nrow(district_panel), 1L)
  if (!is.finite(coverage) || coverage < 0.75) {
    return(list(
      status = "out_of_active_pipeline",
      reason = paste0(
        "Requires validated non-empty geometry for at least 75% of district-panel rows; current coverage is ",
```

```r
        round(100 * coverage, 1), "%.")
    ))
  }

  weights <- build_spatial_weights_for_rows(district_panel, row_index, queen = FALSE, snap =
   ↪ spatial_snap_setting(cfg))
  weights$coverage <- coverage
  weights$panel_scope <- "current_final_matched_panel_non_empty_geometry"
  weights
}

#' rows used by final-panel spatial diagnostics
#'
#' Spatial diagnostics operate on the active `district_panel` target, not on the
#' legacy exploratory geometry object.  The final-panel scope is therefore all
#' rows in the current matched panel with non-empty geometry.
spatial_weight_final_panel_rows <- function(district_panel) {
  if (!inherits(district_panel, "sf")) return(integer())
  geom <- sf::st_geometry(district_panel)
  which(!sf::st_is_empty(geom))
}

spatial_snap_setting <- function(cfg) {
  cfg <- cfg %||% list()
  spatial_cfg <- cfg$spatial_weights %||% list()
  value <- spatial_cfg$snap %||% cfg$spatial_snap %||% NULL
  if (is.null(value) || !length(value) || is.na(value[[1]])) return(NULL)
  value <- suppressWarnings(as.numeric(value[[1]]))
  if (!is.finite(value) || value < 0) stop("Spatial snap must be a finite non-negative
   ↪ number.", call. = FALSE)
  value
}

poly2nb_with_optional_snap <- function(panel, queen, snap = NULL) {
  if (is.null(snap)) return(spdep::poly2nb(panel, queen = queen))
  spdep::poly2nb(panel, queen = queen, snap = snap)
}

#' build spatial weights for selected district-panel rows
#'
#' @return A list with nb, binary matrix, and row-standardized listw objects.
build_spatial_weights_for_rows <- function(district_panel, rows, queen = FALSE, snap = NULL)
 ↪ {
  if (!inherits(district_panel, "sf")) {
    return(list(status = "out_of_active_pipeline", reason = "Requires sf geometry."))
  }
  need_pkg("spdep", "spatial weights")
  need_pkg("sf", "spatial weights")

  rows <- as.integer(rows)
  rows <- rows[is.finite(rows) & rows >= 1L & rows <= nrow(district_panel)]
  if (!length(rows)) {
    return(list(status = "out_of_active_pipeline", reason = "No rows with usable
     ↪ geometry."))
  }
```

```r
geom <- sf::st_geometry(district_panel)
rows <- rows[!sf::st_is_empty(geom[rows])]
if (!length(rows)) {
  return(list(status = "out_of_active_pipeline", reason = "Selected rows have empty
   ↪  geometry."))
}

spatial_warnings <- character()
capture_expected_spatial_warning <- function(expr) {
  withCallingHandlers(
    expr,
    warning = function(w) {
      msg <- conditionMessage(w)
      expected <- grepl("no neighbours|sub-graphs", msg, ignore.case = TRUE)
      if (expected) {
        spatial_warnings <<- unique(c(spatial_warnings, msg))
        invokeRestart("muffleWarning")
      }
    }
  )
}

panel <- district_panel[rows, , drop = FALSE]
nb <- capture_expected_spatial_warning(poly2nb_with_optional_snap(panel, queen = queen,
↪  snap = snap))
W <- capture_expected_spatial_warning(spdep::nb2mat(nb, style = "B", zero.policy = TRUE))
listw <- capture_expected_spatial_warning(spdep::nb2listw(nb, style = "W", zero.policy =
↪  TRUE))

neighbor_counts <- spdep::card(nb)
n_components <- spdep::n.comp.nb(nb)$nc %||% NA_integer_
snap_used <- if (is.null(snap)) NA_real_ else as.numeric(snap)
panel_data <- as.data.frame(sf::st_drop_geometry(panel), stringsAsFactors = FALSE)
identifier_cols <- intersect(
  c("district_panel_id", "state_20", "district_20", "state_std", "district_std"),
  names(panel_data)
)
neighbor_ledger <- data.frame(
  row_index = rows,
  n_neighbors = neighbor_counts,
  panel_data[identifier_cols],
  check.names = FALSE,
  stringsAsFactors = FALSE
)

out <- list(
  status = "constructed",
  contiguity = if (isTRUE(queen)) "queen" else "rook",
  style = "W",
  matrix_style = "B",
  zero_policy = TRUE,
  row_index = rows,
  nb = nb,
  W = W,
  listw = listw,
```

```r
    neighbor_counts = neighbor_counts,
    neighbor_ledger = neighbor_ledger,
    n = length(rows),
    n_islands = sum(neighbor_counts == 0L),
    mean_neighbors = mean(neighbor_counts),
    n_subgraphs = n_components,
    snap = snap_used,
    snap_investigation_needed = sum(neighbor_counts == 0L) > 0L || (!is.na(n_components) &&
     ↪  n_components > 1L),
    warnings = spatial_warnings
  )
  class(out) <- c("emi_spatial_weights", class(out))
  out
}

#' diagnose spatial weights
#'
diagnose_spatial_weights <- function(district_panel, spatial_weights, cfg) {
  if (!inherits(district_panel, "sf")) {
    return(data.frame(
      diagnostic = "spatial_weights",
      status = "out_of_active_pipeline",
      reason = "Requires sf geometry.",
      stringsAsFactors = FALSE
    ))
  }

  comparison <- compare_rook_queen_contiguity(district_panel, snap =
 ↪  spatial_snap_setting(cfg))
  base <- data.frame(
    diagnostic = "spatial_weights",
    status = spatial_weights$status %||% "constructed",
    contiguity = spatial_weights$contiguity %||% NA_character_,
    style = spatial_weights$style %||% NA_character_,
    matrix_style = spatial_weights$matrix_style %||% NA_character_,
    n = spatial_weights$n %||% NA_real_,
    n_islands = spatial_weights$n_islands %||% NA_real_,
    mean_neighbors = spatial_weights$mean_neighbors %||% NA_real_,
    n_subgraphs = spatial_weights$n_subgraphs %||% NA_integer_,
    snap = spatial_weights$snap %||% NA_real_,
    snap_investigation_needed = spatial_weights$snap_investigation_needed %||% NA,
    panel_scope = spatial_weights$panel_scope %||%
     ↪  "current_final_matched_panel_non_empty_geometry",
    warnings = paste(spatial_weights$warnings %||% attr(spatial_weights, "spatial_warnings")
     ↪  %||% character(), collapse = "; "),
    stringsAsFactors = FALSE
  )
  attr(base, "rook_queen_comparison") <- add_spatial_weight_reference(comparison)
  attr(base, "legacy_reference") <- spatial_weight_reference()
  attr(base, "neighbor_counts") <- summarize_neighbor_counts(spatial_weights)
  attr(base, "islands") <- summarize_islands(spatial_weights)
  attr(base, "connectivity") <- summarize_spatial_connectivity(spatial_weights)
  base
}
```

```r
#' compare rook and queen contiguity
#'
#' The legacy Rmd commented out an sfExtras rook-vs-queen check and recorded
#' nearly identical mean neighbor counts (rook 4.780165, queen 4.783471) plus
#' similar run times.  To avoid an extra sfExtras dependency, this project uses
#' the same `spdep::poly2nb()` implementation used by the final rook weights and
#' records elapsed time and average neighbor counts for both choices.
compare_rook_queen_contiguity <- function(district_panel, snap = NULL) {
  if (!inherits(district_panel, "sf")) return(tibble::tibble())
  need_pkg("spdep", "rook/queen contiguity comparison")
  need_pkg("sf", "rook/queen contiguity comparison")
  panel <- district_panel[spatial_weight_final_panel_rows(district_panel), , drop = FALSE]
  if (!nrow(panel)) return(tibble::tibble())

  one <- function(queen) {
    warnings <- character()
    elapsed <- system.time({
      nb <- withCallingHandlers(
        poly2nb_with_optional_snap(panel, queen = queen, snap = snap),
        warning = function(w) {
          msg <- conditionMessage(w)
          if (grepl("no neighbours|sub-graphs", msg, ignore.case = TRUE)) {
            warnings <<- unique(c(warnings, msg))
            invokeRestart("muffleWarning")
          }
        }
      )
    })[["elapsed"]]
    cards <- spdep::card(nb)
    tibble::tibble(
      contiguity = if (isTRUE(queen)) "queen" else "rook",
      n = length(nb),
      mean_neighbors = mean(cards),
      n_islands = sum(cards == 0L),
      n_subgraphs = spdep::n.comp.nb(nb)$nc %||% NA_integer_,
      snap = if (is.null(snap)) NA_real_ else as.numeric(snap),
      panel_scope = "current_final_matched_panel_non_empty_geometry",
      elapsed_seconds = unname(elapsed),
      warnings = paste(warnings, collapse = "; ")
    )
  }

  safe_bind_rows(list(one(FALSE), one(TRUE)))
}


spatial_weight_reference <- function() {
  data.frame(
    contiguity = c("rook", "queen"),
    legacy_method = c(
      "sfExtras::st_rook() timing comment; final weights use spdep::poly2nb(queen = FALSE)",
      "sfExtras::st_queen() timing comment; benchmark uses spdep::poly2nb(queen = TRUE)"
    ),
    legacy_mean_neighbors = c(4.780165, 4.783471),
```

```r
    legacy_elapsed_note = c("legacy comment recorded similar run time to queen", "legacy
     ↪  comment recorded similar run time to rook"),
    interpretation = "Current means are intentionally computed on the active final matched
     ↪  panel with non-empty geometry; they may differ from legacy exploratory sfExtras
     ↪  objects, but they now match the panel used for current spatial diagnostics.",
    stringsAsFactors = FALSE
  )
}

add_spatial_weight_reference <- function(comparison) {
  comparison <- as.data.frame(comparison, stringsAsFactors = FALSE)
  if (!nrow(comparison)) return(comparison)
  ref <- spatial_weight_reference()[c("contiguity", "legacy_mean_neighbors")]
  out <- merge(comparison, ref, by = "contiguity", all.x = TRUE, sort = FALSE)
  out$mean_neighbor_delta_from_legacy <- out$mean_neighbors - out$legacy_mean_neighbors
  out$pct_delta_from_legacy <- 100 * out$mean_neighbor_delta_from_legacy /
 ↪  out$legacy_mean_neighbors
  out
}

#' summarize islands
#'
summarize_islands <- function(spatial_weights) {
  ledger <- summarize_neighbor_counts(spatial_weights)
  if (!nrow(ledger)) return(ledger)
  ledger[ledger$n_neighbors == 0L, , drop = FALSE]
}

#' summarize neighbor counts
#'
summarize_neighbor_counts <- function(spatial_weights) {
  if (!inherits(spatial_weights, "emi_spatial_weights")) return(tibble::tibble())
  ledger <- spatial_weights$neighbor_ledger %||% data.frame(
    row_index = spatial_weights$row_index,
    n_neighbors = spatial_weights$neighbor_counts
  )
  tibble::as_tibble(ledger)
}

#' Summarize graph connectivity and whether `poly2nb()` snap needs review
summarize_spatial_connectivity <- function(spatial_weights) {
  if (!inherits(spatial_weights, "emi_spatial_weights")) return(tibble::tibble())
  tibble::tibble(
    n = spatial_weights$n,
    n_islands = spatial_weights$n_islands,
    n_subgraphs = spatial_weights$n_subgraphs,
    snap = spatial_weights$snap,
    snap_investigation_needed = isTRUE(spatial_weights$snap_investigation_needed),
    recommendation = if (isTRUE(spatial_weights$snap_investigation_needed)) {
      "Inspect geometry validity and sensitivity to a documented poly2nb snap value before
       ↪  interpreting spatial results."
    } else {
      "No island/subgraph-driven snap investigation is currently indicated."
    }
  )
```

```r
}

#' save spatial weight diagnostics
#'
save_spatial_weight_diagnostics <- function(diagnostics, dir =
  "outputs/diagnostics/extended/spatial") {
  dir.create(dir, recursive = TRUE, showWarnings = FALSE)
  paths <- c(
    summary = write_diagnostic_csv(as.data.frame(diagnostics), file.path(dir,
      "spatial_weights_summary.csv")),
    rook_queen_comparison = write_diagnostic_csv(attr(diagnostics, "rook_queen_comparison")
      %||% data.frame(), file.path(dir, "rook_queen_contiguity_comparison.csv")),
    legacy_reference = write_diagnostic_csv(attr(diagnostics, "legacy_reference") %||%
      data.frame(), file.path(dir, "spatial_weights_legacy_reference.csv")),
    neighbor_counts = write_diagnostic_csv(attr(diagnostics, "neighbor_counts") %||%
      data.frame(), file.path(dir, "spatial_weights_neighbor_counts.csv")),
    islands = write_diagnostic_csv(attr(diagnostics, "islands") %||% data.frame(),
      file.path(dir, "spatial_weights_islands.csv")),
    connectivity = write_diagnostic_csv(attr(diagnostics, "connectivity") %||% data.frame(),
      file.path(dir, "spatial_weights_connectivity.csv"))
  )
  output_manifest(paths)
}
```

## Reusable IV specification and estimation

Source: R/iv/build_iv_formulas.R

```r
#' make iv formula
#'
make_iv_formula <- function(dep, endog, instruments, controls = NULL, fixed_effects = NULL)
  {
  stats::as.formula(paste(
    dep,
    "~",
    paste(c(endog, controls, fixed_effects), collapse = " + "),
    "|",
    paste(c(instruments, controls, fixed_effects), collapse = " + ")
  ))
}

baseline_iv_controls <- function() {
  c(
    "consumption_0708", "gini_cons_0708",
    "pct_urban", "avg_hh_size", "dependency_ratio",
    "pct_fem_head",
    "pct_hindu", "pct_muslim",
    "pct_st", "pct_sc", "pct_obc",
    "pct_small_land", "pct_medium_land", "pct_large_land",
    "pct_head_lit_to_primary", "pct_head_secondary_plus"
  )
}

#' build iv formulas
#'
```

```r
build_iv_formulas <- function(cfg) {
  controls <- baseline_iv_controls()
  list(
    consumption = make_iv_formula(
      "consumption_pct_change",
      "EMIE",
      "wavg_ling_degrees",
      controls
    ),
    gini = make_iv_formula(
      "gini_change",
      "EMIE",
      "wavg_ling_degrees",
      controls
    )
  )
}
```

## Multicollinearity and spatial autocorrelation diagnostics

Source: `R/diagnostics/diagnose_multicollinearity.R`

```r
#' Structural-regressor formula for multicollinearity diagnostics
#'
#' For `ivreg` models, multicollinearity is diagnosed on the stage-two
#' structural regressors, not on the instrument matrix. The `ivreg` package
#' exposes this component through its formula/model-matrix methods.
multicollinearity_formula <- function(model) {
  if (inherits(model, "ivreg")) {
    return(stats::formula(model, component = "regressors"))
  }
  stats::formula(model)
}

#' Structural-regressor design matrix
multicollinearity_design_matrix <- function(model) {
  if (inherits(model, "ivreg")) {
    return(stats::model.matrix(model, component = "regressors"))
  }
  stats::model.matrix(model)
}

#' Model used for term-aware VIF/GVIF diagnostics
#'
#' `car::vif()` reports ordinary VIFs for one-degree-of-freedom terms and
#' generalized VIFs for factors or other multi-column terms. For an IV model,
#' fit an auxiliary OLS model with the same response and structural regressors.
#' VIFs describe collinearity in that regressor design, so the excluded
#' instrument matrix is deliberately outside this diagnostic.
multicollinearity_vif_model <- function(model) {
  if (!inherits(model, "ivreg")) return(model)
  stats::lm(
    multicollinearity_formula(model),
    data = stats::model.frame(model),
    weights = tryCatch(stats::weights(model), error = function(e) NULL),
    na.action = stats::na.exclude
```

```r
  )
}

#' Normalize `car::vif()` output to a stable data-frame contract
normalize_vif_output <- function(x, model_scope) {
  if (is.null(x) || !length(x)) return(data.frame())
  if (is.matrix(x)) {
    out <- data.frame(
      term = rownames(x),
      gvif = as.numeric(x[, "GVIF"]),
      df = as.integer(x[, "Df"]),
      gvif_scaled = as.numeric(x[, "GVIF^(1/(2*Df))"]),
      stringsAsFactors = FALSE
    )
    out$vif <- ifelse(out$df == 1L, out$gvif, NA_real_)
  } else {
    out <- data.frame(
      term = names(x),
      gvif = as.numeric(x),
      df = 1L,
      gvif_scaled = sqrt(as.numeric(x)),
      vif = as.numeric(x),
      stringsAsFactors = FALSE
    )
  }
  out$model_scope <- model_scope
  out$status <- "estimated"
  out$reason <- NA_character_
  out[c("term", "model_scope", "df", "vif", "gvif", "gvif_scaled", "status", "reason")]
}

#' Compute term-aware VIF/GVIF diagnostics
compute_vif_if_applicable <- function(model) {
  scope <- if (inherits(model, "ivreg")) "ivreg_structural_regressors" else
  ↪  "model_regressors"
  if (!requireNamespace("car", quietly = TRUE)) {
    return(data.frame(
      term = NA_character_, model_scope = scope, df = NA_integer_,
      vif = NA_real_, gvif = NA_real_, gvif_scaled = NA_real_,
      status = "unavailable", reason = "Package 'car' is not installed.",
      stringsAsFactors = FALSE
    ))
  }
  tryCatch(
    normalize_vif_output(car::vif(multicollinearity_vif_model(model)), scope),
    error = function(e) data.frame(
      term = NA_character_, model_scope = scope, df = NA_integer_,
      vif = NA_real_, gvif = NA_real_, gvif_scaled = NA_real_,
      status = "unavailable", reason = conditionMessage(e),
      stringsAsFactors = FALSE
    )
  )
}

multicollinearity_model_names <- function(iv_models) {
```

```
  models <- if (is.list(iv_models) && !inherits(iv_models, c("lm", "ivreg"))) iv_models else
↪   list(iv_models)
  model_names <- names(models)
  if (is.null(model_names)) model_names <- rep("", length(models))
  missing_names <- !nzchar(model_names)
  model_names[missing_names] <- paste0("model_", which(missing_names))
  list(models = models, names = model_names)
}

#' Diagnose one model's structural-regressor design
single_model_multicollinearity <- function(model, model_name) {
  scope <- if (inherits(model, "ivreg")) "ivreg_structural_regressors" else
↪   "model_regressors"
  design <- tryCatch({
    X <- multicollinearity_design_matrix(model)
    data.frame(
      model = model_name,
      diagnostic = "design_matrix",
      term = NA_character_,
      model_scope = scope,
      n = nrow(X),
      rank = qr(X)$rank,
      columns = ncol(X),
      kappa = kappa(X, exact = TRUE),
      df = NA_integer_,
      vif = NA_real_,
      gvif = NA_real_,
      gvif_scaled = NA_real_,
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  }, error = function(e) {
    data.frame(
      model = model_name,
      diagnostic = "design_matrix",
      term = NA_character_, model_scope = scope,
      n = NA_integer_, rank = NA_integer_, columns = NA_integer_, kappa = NA_real_,
      df = NA_integer_, vif = NA_real_, gvif = NA_real_, gvif_scaled = NA_real_,
      status = "out_of_active_pipeline", reason = conditionMessage(e),
      stringsAsFactors = FALSE
    )
  })

  vif <- compute_vif_if_applicable(model)
  if (!nrow(vif)) return(design)
  vif$model <- model_name
  vif$diagnostic <- "vif"
  vif$n <- NA_integer_
  vif$rank <- NA_integer_
  vif$columns <- NA_integer_
  vif$kappa <- NA_real_
  vif <- vif[names(design)]
  rbind(design, vif)
}
```

```r
#' Diagnose multicollinearity
#'
#' @return A data frame containing a design-matrix summary row and term-level
#'   VIF/GVIF rows for every supplied model. Factor terms use GVIF and the
#'   dimension-adjusted `GVIF^(1/(2*Df))`; IV models use structural regressors.
diagnose_multicollinearity <- function(district_panel, iv_models, cfg) {
  named <- multicollinearity_model_names(iv_models)
  safe_bind_rows(Map(single_model_multicollinearity, named$models, named$names))
}

#' Save public multicollinearity diagnostics
save_multicollinearity_diagnostics <- function(diagnostics, path =
↪  "outputs/diagnostics/public/multicollinearity_diagnostics.csv") {
  write_diagnostic_csv(diagnostics, path)
}
```

### Automated figure generation

Source: `R/output/make_figures.R`

```r
figure_spec <- function(name, file, title, subtitle = NULL, kind = "status", variable =
↪  NULL, sources = NULL, inputs = NULL) {
  out <- list(
    name = name,
    file = file,
    title = title,
    subtitle = subtitle,
    kind = kind,
    variable = variable,
    sources = sources,
    inputs = inputs
  )
}

has_sf_geometry <- function(x) {
  inherits(x, "sf") && !is.null(attr(x, "sf_column")) && attr(x, "sf_column") %in% names(x)
}

sf_geometry_coverage <- function(x) {
  if (!has_sf_geometry(x) || !nrow(x)) return(0)
  mean(!sf::st_is_empty(sf::st_geometry(x)))
}


#' make figures
#'
#' @return A named list of figure specifications consumed by save_figures().
make_figures <- function(district_panel, raw_ilo_figures, cfg, boundaries_2020 = NULL,
↪  iv_models = NULL) {
  required_variables <- c(
    "EMIE",
    "consumption_pct_change",
    "pct_pucca",
    "pct_head_secondary_plus",
    "region",
```

```r
  "wavg_ling_degrees"
)

out <- list(
  fig_ilo_trends = figure_spec(
    "fig_ilo_trends",
    "fig_ilo_trends.png",
    "ILO labor market indicators",
    "Composed from the archived ILO figure assets.",
    kind = "ilo_collage",
    sources = raw_ilo_figures
  ),
  district_carveouts_shifts = figure_spec(
    "district_carveouts_shifts",
    "district_carveouts_shifts.png",
    "District carve-outs and shifts",
    kind = "district_carveouts_shifts"
  ),
  poster_emie_expected_values = figure_spec(
    "poster_emie_expected_values",
    "poster_emie_expected_values.png",
    "Adjusted consumption growth across EMI exposure",
    "Average counterfactual predictions at observed EMIE percentiles.",
    kind = "emie_expected_values"
  )
)

missing_vars <- setdiff(required_variables, names(as.data.frame(district_panel)))
geometry_ok <- has_sf_geometry(district_panel) &&
↪ is.finite(sf_geometry_coverage(district_panel)) && sf_geometry_coverage(district_panel)
↪ >= 0.75
maps_available <- !length(missing_vars) && geometry_ok

map_specs <- list(
  map_emi_exposure = figure_spec("map_emi_exposure", "map_emi_exposure.png", "EMI
   ↪ Exposure", kind = if (maps_available) "map" else "status", variable = "EMIE"),
  map_consumption_growth = figure_spec("map_consumption_growth",
   ↪ "map_consumption_growth.png", "% Change in Consumption", kind = if (maps_available)
   ↪ "map" else "status", variable = "consumption_pct_change"),
  map_pucca = figure_spec("map_pucca", "map_pucca.png", "% Pucca Homes", kind = if
   ↪ (maps_available) "map" else "status", variable = "pct_pucca"),
  map_education = figure_spec("map_education", "map_education.png", "% HH Head w/ Sec.+",
   ↪ kind = if (maps_available) "map" else "status", variable =
   ↪ "pct_head_secondary_plus"),
  map_region = figure_spec("map_region", "map_region.png", "Region", kind = if
   ↪ (maps_available) "map" else "status", variable = "region"),
  map_linguistic_distance = figure_spec("map_linguistic_distance",
   ↪ "map_linguistic_distance.png", "Linguistic Distance", kind = if (maps_available)
   ↪ "map" else "status", variable = "wavg_ling_degrees"),
  collage_main_maps = figure_spec(
    "collage_main_maps",
    "collage_main_maps.png",
    "Main district-level map inputs",
    kind = "collage",
    inputs = c("map_emi_exposure", "map_consumption_growth", "map_pucca", "map_education")
```

```
    ),
    collage_iv_region_maps = figure_spec(
      "collage_iv_region_maps",
      "collage_iv_region_maps.png",
      "Instrument and region map inputs",
      kind = "collage",
      inputs = c("map_region", "map_linguistic_distance")
    )
  )

  if (!maps_available && !identical(cfg$mode, "final")) {
    # Draft-mode diagnostics live outside outputs/figures/main and are explicitly
    # labeled as diagnostics by figure_output_dir().
    map_specs <- map_specs[c("map_emi_exposure", "map_consumption_growth")]
  }

  map_input_failures <- character()
  if (!maps_available && identical(cfg$mode, "final")) {
    map_input_failures <- c(
      if (length(missing_vars)) paste0("Missing map variables: ", paste(missing_vars,
       ↪ collapse = ", ")),
      paste0("Geometry coverage: ", round(100 * sf_geometry_coverage(district_panel), 1),
       ↪ "%")
    )
  }

  out <- c(out, map_specs)

  if (length(map_input_failures)) {
    attr(out, "map_input_failures") <- map_input_failures
  }
  attr(out, "district_panel") <- district_panel
  attr(out, "boundaries_2020") <- boundaries_2020
  attr(out, "iv_models") <- iv_models
  out
}
```

### Publication-quality regression and summary tables

Source: R/output/make_tables.R

```
table_status_failures <- function(x, cfg, label) {
  df <- as.data.frame(x)
  ok_status <- c("mapped", "estimated")
  if (!is_final_mode(cfg) || !"status" %in% names(df)) return(character())
  bad <- !is.na(df$status) & !df$status %in% ok_status
  if (!any(bad)) return(character())
  reasons <- character()
  if ("reason" %in% names(df)) reasons <-
   ↪ unique(stats::na.omit(as.character(df$reason[bad])))
  suffix <- if (length(reasons)) paste0(" Reasons: ", paste(reasons, collapse = "; ")) else
↪   ""
  paste0("Final table generation requires completed model output for ", label, ".", suffix)
}
```

```r
public_numeric_stats <- function(df, meta, cost_vars = character(), count_vars =
 ↪  character()) {
  df <- as.data.frame(df)
  rows <- lapply(seq_len(nrow(meta)), function(i) {
    v <- meta$var[[i]]
    if (!v %in% names(df)) return(NULL)
    x <- suppressWarnings(as.numeric(as.character(df[[v]])))
    ok <- is.finite(x)
    if (!any(ok)) return(NULL)
    stats <- c(
      Min = min(x[ok], na.rm = TRUE),
      `1Q` = unname(stats::quantile(x[ok], .25, na.rm = TRUE)),
      Med = stats::median(x[ok], na.rm = TRUE),
      `3Q` = unname(stats::quantile(x[ok], .75, na.rm = TRUE)),
      Max = max(x[ok], na.rm = TRUE),
      Mean = mean(x[ok], na.rm = TRUE),
      SD = stats::sd(x[ok], na.rm = TRUE)
    )
    z <- data.frame(var = v, label = meta$label[[i]], N = sum(ok), t(stats), check.names =
 ↪  FALSE, stringsAsFactors = FALSE)
    if ("desc" %in% names(meta)) z$desc <- meta$desc[[i]]
    z
  })
  out <- safe_bind_rows(rows)
  if (!nrow(out)) return(data.frame(var = character(), label = character(), N = integer()))
  for (nm in intersect(c("Min", "1Q", "Med", "3Q", "Max", "Mean", "SD"), names(out))) {
    x <- suppressWarnings(as.numeric(out[[nm]]))
    out[[nm]] <- ifelse(
      out$var %in% count_vars,
      formatC(round(x), format = "f", big.mark = ",", digits = 0),
      ifelse(out$var %in% cost_vars, formatC(x, format = "f", big.mark = ",", digits = 2),
       ↪  sprintf("%.2f", x))
    )
  }
  out
}

public_categorical_stats <- function(df, meta) {
  df <- as.data.frame(df)
  rows <- lapply(seq_len(nrow(meta)), function(i) {
    v <- meta$var[[i]]
    if (!v %in% names(df)) return(NULL)
    x <- df[[v]]
    x <- x[!is.na(x)]
    if (!length(x)) return(NULL)
    freq <- table(x)
    data.frame(
      var = v,
      label = meta$label[[i]],
      N = length(x),
      Values = paste(names(freq), collapse = ", "),
      Mode = names(freq)[which.max(freq)],
      `% Mode` = round(max(freq) / sum(freq) * 100, 1),
      `Least Freq.` = names(freq)[which.min(freq)],
      `% Least Freq.` = round(min(freq) / sum(freq) * 100, 1),
```

```r
      check.names = FALSE,
      stringsAsFactors = FALSE
    )
  })
  out <- safe_bind_rows(rows)
  if (!nrow(out)) data.frame(var = character(), label = character(), N = integer()) else out
}

summary_group_row <- function(label, columns) {
  row <- as.list(stats::setNames(rep(NA_character_, length(columns)), columns))
  if ("var" %in% columns) row$var <- paste0(".group_", gsub("[^A-Za-z0-9]+", "_", label))
  if ("label" %in% columns) row$label <- label
  as.data.frame(row, check.names = FALSE, stringsAsFactors = FALSE)
}

insert_summary_group <- function(df, label, before_var) {
  if (!nrow(df) || !"var" %in% names(df)) return(df)
  pos <- match(before_var, df$var)
  if (is.na(pos)) return(df)
  rbind(
    if (pos > 1L) df[seq_len(pos - 1L), , drop = FALSE] else df[0L, , drop = FALSE],
    summary_group_row(label, names(df)),
    df[pos:nrow(df), , drop = FALSE]
  )
}

significance_stars <- function(p) {
  ifelse(is.finite(p) & p < 0.001, "***",
    ifelse(is.finite(p) & p < 0.01, "**",
      ifelse(is.finite(p) & p < 0.05, "*", "")))
}

format_estimate <- function(estimate, p.value = NA_real_) {
  ifelse(is.finite(estimate), paste0(sprintf("%.3f", estimate),
   ↪ significance_stars(p.value)), NA_character_)
}

format_se <- function(std.error) {
  ifelse(is.finite(std.error), sprintf("(%.3f)", std.error), NA_character_)
}

regression_display_table <- function(terms, estimates, std_errors, p_values, outcome_label,
 ↪ gof = NULL) {
  rows <- safe_bind_rows(lapply(seq_along(terms), function(i) {
    data.frame(
      Term = c(terms[[i]], ""),
      value = c(format_estimate(estimates[[i]], p_values[[i]]), format_se(std_errors[[i]])),
      stringsAsFactors = FALSE
    )
  }))
  names(rows)[names(rows) == "value"] <- outcome_label
  if (!is.null(gof) && nrow(gof)) {
    names(gof) <- names(rows)
    rows <- rbind(rows, gof)
  }
```

```r
  rows
}

first_finite_scalar <- function(value) {
  value <- suppressWarnings(as.numeric(value))
  value <- value[is.finite(value)]
  if (length(value)) value[[1]] else NA_real_
}

format_gof_number <- function(value, digits = 3L, integer = FALSE) {
  value <- first_finite_scalar(value)
  if (!is.finite(value)) return("")
  if (isTRUE(integer)) sprintf("%.0f", value) else sprintf(paste0("%.", digits, "f"), value)
}

model_gof_frame <- function(model) {
  if (is.null(model) || is_model_status_payload(model)) return(data.frame())
  sm <- tryCatch(summary(model), error = function(e) NULL)
  fstat <- first_finite_scalar(tryCatch(sm$fstatistic, error = function(e) NA_real_))
  data.frame(
    nobs = first_finite_scalar(tryCatch(stats::nobs(model), error = function(e) NA_real_)),
    r.squared = first_finite_scalar(tryCatch(sm$r.squared, error = function(e) NA_real_)),
    adj.r.squared = first_finite_scalar(tryCatch(sm$adj.r.squared, error = function(e)
     ↪  NA_real_)),
    sigma = first_finite_scalar(tryCatch(sm$sigma, error = function(e) NA_real_)),
    statistic = fstat,
    waldtest = fstat,
    check.names = FALSE,
    stringsAsFactors = FALSE
  )
}

model_gof_rows <- function(model, outcome_label) {
  gof <- model_gof_frame(model)
  if (!nrow(gof)) return(data.frame())
  data.frame(
    Term = c("Observations", "R-squared", "Adjusted R-squared", "Model's F-Statistic"),
    value = c(
      format_gof_number(gof$nobs, integer = TRUE),
      format_gof_number(gof$r.squared),
      format_gof_number(gof$adj.r.squared),
      format_gof_number(gof$statistic, digits = 2L)
    ),
    check.names = FALSE,
    stringsAsFactors = FALSE
  ) |>
    stats::setNames(c("Term", outcome_label))
}

probit_gof_rows <- function(selection_model, n, outcome_label) {
  rows <- data.frame(
    Term = "Observations",
    value = format_gof_number(n, integer = TRUE),
    stringsAsFactors = FALSE
  )
```

```r
  # The active participation model is fit with survey::svyglm(), which is
  # design-based rather than maximum-likelihood.  Likelihood-based quantities
  # such as log-likelihood and McFadden's pseudo-R^2 are therefore deliberately
  # omitted for svyglm objects; calling stats::logLik() on svyglm emits the
  # survey package warning that the model was not fitted by maximum likelihood.
  # If this helper is reused for an ordinary glm in diagnostics, the usual ML
  # summary rows are still meaningful and are reported.
  if (inherits(selection_model, "glm") && !inherits(selection_model, "svyglm")) {
    loglik <- tryCatch(stats::logLik(selection_model), error = function(e) NA_real_)
    null_dev <- tryCatch(selection_model$null.deviance, error = function(e) NA_real_)
    dev <- tryCatch(selection_model$deviance, error = function(e) NA_real_)
    pseudo_r2 <- if (is.finite(first_finite_scalar(null_dev)) &&
↪   first_finite_scalar(null_dev) > 0 && is.finite(first_finite_scalar(dev))) {
      1 - first_finite_scalar(dev) / first_finite_scalar(null_dev)
    } else {
      NA_real_
    }
    rows <- rbind(
      rows,
      data.frame(Term = "Log Likelihood", value = format_gof_number(loglik, digits = 2L),
       ↪  stringsAsFactors = FALSE),
      data.frame(Term = "McFadden pseudo-R-squared", value = format_gof_number(pseudo_r2),
       ↪  stringsAsFactors = FALSE)
    )
  }
  stats::setNames(rows, c("Term", outcome_label))
}

table_status_row <- function(model, status = "unavailable", reason = NA_character_) {
  data.frame(
    model = model,
    term = NA_character_,
    estimate = NA_real_,
    std.error = NA_real_,
    statistic = NA_real_,
    p.value = NA_real_,
    status = status,
    reason = reason,
    stringsAsFactors = FALSE
  )
}

table_first_existing_column <- function(df, candidates) {
  hit <- intersect(candidates, names(df))
  if (length(hit)) hit[[1]] else NA_character_
}

table_column_or_na <- function(df, column) {
  if (length(column) != 1L || is.na(column) || !column %in% names(df)) {
    return(rep(NA_real_, nrow(df)))
  }
  suppressWarnings(as.numeric(df[[column]]))
}
```

```
coefficient_frame <- function(model, vcov_matrix = NULL) {
  estimates <- tryCatch(stats::coef(model), error = function(e) NULL)
  if (is.null(estimates) || !length(estimates)) return(data.frame())

  terms <- names(estimates)
  if (is.null(terms) || !length(terms)) terms <- paste0("term_", seq_along(estimates))
  estimates <- suppressWarnings(as.numeric(estimates))

  vc <- vcov_matrix
  if (is.null(vc)) vc <- tryCatch(stats::vcov(model), error = function(e) NULL)
  se <- rep(NA_real_, length(estimates))
  if (!is.null(vc) && length(dim(vc)) == 2L && all(dim(vc) >= length(estimates))) {
    diag_vc <- suppressWarnings(as.numeric(diag(vc)))
    vc_terms <- rownames(vc)
    if (!is.null(vc_terms) && length(vc_terms)) {
      matched <- match(terms, vc_terms)
      ok <- !is.na(matched) & matched <= length(diag_vc)
      se[ok] <- sqrt(pmax(diag_vc[matched[ok]], 0))
    } else {
      se <- sqrt(pmax(diag_vc[seq_along(estimates)], 0))
    }
  }

  statistic <- estimates / se
  statistic[!is.finite(statistic)] <- NA_real_
  df_resid <- tryCatch(stats::df.residual(model), error = function(e) NA_real_)
  p_value <- if (is.finite(df_resid) && df_resid > 0) {
    2 * stats::pt(abs(statistic), df = df_resid, lower.tail = FALSE)
  } else {
    2 * stats::pnorm(abs(statistic), lower.tail = FALSE)
  }
  p_value[!is.finite(p_value)] <- NA_real_

  out <- data.frame(
    Estimate = estimates,
    `Std. Error` = se,
    statistic = statistic,
    `Pr(>|t|)` = p_value,
    check.names = FALSE,
    stringsAsFactors = FALSE
  )
  rownames(out) <- terms
  out
}

plain_model_coefficients <- function(model) {
  coefficient_frame(model)
}

clustered_model_coefficients <- function(model) {
  tryCatch({
    vc_fun <- clustered_model_vcov(model)
    if (is.null(vc_fun)) return(data.frame())
    vc <- vc_fun(model)
    coefficient_frame(model, vc)
```

```
  }, error = function(e) data.frame())
}

filter_table_model <- function(df, candidates) {
  if (!"model" %in% names(df)) return(df)
  out <- df[df$model %in% candidates, , drop = FALSE]
  if (nrow(out)) out else df[0L, , drop = FALSE]
}

is_model_status_payload <- function(x) {
  is.list(x) &&
    !inherits(x, c("lm", "ivreg")) &&
    !is.null(x$status) &&
    length(x$status) == 1L &&
    (is.character(x$status) || is.factor(x$status))
}

model_status_reason <- function(x) {
  reason <- x$reason
  if (is.null(reason) || !length(reason) || all(is.na(reason))) NA_character_ else
   ↪ as.character(reason[[1]])
}

tidy_iv_models <- function(iv_models) {
  if (!is.list(iv_models) || inherits(iv_models, c("lm", "ivreg"))) iv_models <- list(model
   ↪ = iv_models)
  safe_bind_rows(lapply(names(iv_models), function(name) {
    model <- iv_models[[name]]
    if (is_model_status_payload(model)) {
      return(table_status_row(name, as.character(model$status[[1]]),
       ↪ model_status_reason(model)))
    }
    coefs <- clustered_model_coefficients(model)
    if (!nrow(coefs)) coefs <- plain_model_coefficients(model)
    if (!nrow(coefs)) {
      return(table_status_row(name, "unavailable", "Model coefficients are unavailable."))
    }

    estimate_col <- table_first_existing_column(coefs, c("Estimate", "estimate"))
    se_col <- table_first_existing_column(coefs, c("Std. Error", "std.error", "Std.Error"))
    statistic_col <- table_first_existing_column(coefs, c("t value", "z value", "t", "z",
↪   "statistic"))
    p_col <- table_first_existing_column(coefs, c("Pr(>|t|)", "Pr(>|z|)", "p.value", "p",
↪   "P>|t|", "P>|z|"))

    if (is.na(estimate_col)) {
      return(table_status_row(name, "unavailable", paste("Model coefficient table lacks an
       ↪ estimate column. Columns:", paste(names(coefs), collapse = ", "))))
    }

    data.frame(
      model = name,
      term = rownames(coefs),
      estimate = table_column_or_na(coefs, estimate_col),
      std.error = table_column_or_na(coefs, se_col),
```

```r
      statistic = table_column_or_na(coefs, statistic_col),
      p.value = table_column_or_na(coefs, p_col),
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  }))
}

clustered_model_vcov <- function(model) {
  cluster <- attr(model, "cluster_state")
  if (is.null(cluster)) return(NULL)
  cluster <- as.vector(cluster)
  if (length(cluster) != stats::nobs(model)) return(NULL)
  if (sum(!is.na(cluster)) < 2L || length(unique(cluster[!is.na(cluster)])) <= 1L)
   ↪  return(NULL)
  if (!requireNamespace("sandwich", quietly = TRUE)) return(NULL)
  force(cluster)
  function(x) sandwich::vcovCL(x, cluster = cluster)
}

#' make tables
#'
#' @return A named list of data frames consumed by save_tables().
make_tables <- function(selection_data, ame_results, district_panel, iv_models,
 ↪  first_stage_tests, cfg, selection_model = NULL) {
  cons_iv <- tidy_iv_models(iv_models)
  cons_iv_required <- filter_table_model(cons_iv, c("consumption", "baseline"))
  table_failures <- c(
    table_status_failures(ame_results, cfg, "average marginal effects"),
    table_status_failures(first_stage_tests, cfg, "first-stage regression"),
    table_status_failures(cons_iv_required, cfg, "second-stage IV regression")
  )

  fs_cons <- make_first_stage_table(first_stage_tests, cfg)
  cons_iv_table <- make_second_stage_table(iv_models)
  fs_model <- first_stage_table_model(iv_models, district_panel)
  if (!is.null(fs_model$model)) {
    attr(fs_cons, "table_model") <- fs_model$model
    attr(fs_cons, "table_vcov") <- fs_model$vcov
    attr(fs_cons, "table_add_rows") <- fs_model$add_rows
  }
  cons_model <- second_stage_table_model(iv_models)
  if (!is.null(cons_model$model)) {
    attr(cons_iv_table, "table_model") <- cons_model$model
    attr(cons_iv_table, "table_vcov") <- cons_model$vcov
  }

  out <- list(
    selection_n = data.frame(n = nrow(as.data.frame(selection_data))),
    sum_tbl_probit_quant = make_selection_summary_numeric_table(selection_data),
    sum_tbl_probit_cat = make_selection_summary_categorical_table(selection_data),
    probit_mfx = make_probit_ame_table(ame_results, nrow(as.data.frame(selection_data)),
     ↪  selection_model),
    sum_tbl_iv = make_iv_summary_table(district_panel),
```

```r
    fs_cons = fs_cons,
    cons_iv = cons_iv_table,
    ame_results = as.data.frame(ame_results),
    first_stage = as.data.frame(first_stage_tests)
  )
  table_failures <- c(table_failures, attr(out$fs_cons, "table_input_failures"))
  table_failures <- unique(stats::na.omit(table_failures))
  if (length(table_failures)) attr(out, "table_input_failures") <- table_failures
  out
}

make_selection_summary_numeric_table <- function(selection_data) {
  meta <- data.frame(
    var = c("AGE", "HH_SIZE", "ENROLLMENT_COST", "dmean_num_IS_EDU_FREE",
     ↪ "dmean_num_TUTION_FEE_WAIVED", "dmean_num_RECD_SCHOLARSHIP_STIPEND",
     ↪ "dmean_num_RECD_TXT_BOOKS", "dmean_num_RECD_STATIONERY",
     ↪ "dmean_num_MID_DAY_MEAL_ETC_RECD", "dmean_num_ENROLLMENT_COST"),
    label = c("Age", "Household size", "Enrollment cost (Rs.)", "Educ. free available? (Yes
     ↪ = 1)", "Tuition waived?", "Scholarship/Stipend?", "Textbooks received?", "Stationery
     ↪ received?", "Mid-day meal or more received?", "Avg. district enrollment cost"),
    stringsAsFactors = FALSE
  )
  out <- public_numeric_stats(selection_data, meta, cost_vars = "ENROLLMENT_COST")
  insert_summary_group(out, "District-level aggregates:", "dmean_num_IS_EDU_FREE")
}

make_selection_summary_categorical_table <- function(selection_data) {
  meta <- data.frame(
    var = c("SEX", "RELIGION", "SOCIAL_GROUP", "SECTOR", "DIST_FROM_NEAREST_PRIMARY_CLASS",
     ↪ "father_educ"),
    label = c("Sex", "Religion", "Social group", "Urban", "Distance of nearest primary
     ↪ class", "Father's education"),
    stringsAsFactors = FALSE
  )
  public_categorical_stats(selection_data, meta)
}

make_probit_ame_table <- function(ame_results, n = NA_integer_, selection_model = NULL) {
  native_ame <- attr(ame_results, "marginaleffects_object", exact = TRUE)
  out <- as.data.frame(ame_results, stringsAsFactors = FALSE)
  if (!"Term" %in% names(out)) out$Term <- out$term
  table <- data.frame(
    Term = out$Term,
    Estimate = format_estimate(
      suppressWarnings(as.numeric(out$estimate)),
      suppressWarnings(as.numeric(out$p.value))
    ),
    `Std. Error` = format_se(suppressWarnings(as.numeric(out$std.error))),
    check.names = FALSE,
    stringsAsFactors = FALSE
  )
  if (!is.null(native_ame)) {
    attr(table, "marginaleffects_object") <- native_ame
    attr(table, "marginaleffects_n") <- n
  }
```

```r
  if (!is.null(selection_model)) {
    attr(table, "selection_model") <- selection_model
  }
  table
}

make_iv_summary_table <- function(district_panel) {
  meta <- data.frame(
    var = c(
      "wavg_ling_degrees", "EMIE",
      "npeople_0708", "consumption_0708", "gini_cons_0708", "pct_urban", "avg_hh_size",
      "dependency_ratio", "pct_fem_head", "pct_hindu", "pct_muslim", "pct_other_religion",
      "pct_st", "pct_sc", "pct_obc", "pct_small_land", "pct_medium_land", "pct_large_land",
      "pct_head_illiterate", "pct_head_lit_to_primary", "pct_head_secondary_plus",
      ↪ "pct_pucca",
      "npeople_1718", "consumption_1718", "gini_cons_1718",
      "consumption_pct_change", "gini_change"
    ),
    label = c(
      "Ling. Distance",
      "EMIE", "Population", "Consumption", "Gini of Consumption", "Pct. Urban", "Avg. HH
      ↪ Size",
      "Dependency Ratio × 100", "Pct. Female Head", "Pct. Hindu", "Pct. Muslim", "Pct.
      ↪ Other",
      "Pct. ST", "Pct. SC", "Pct. OBC", "Pct. Small Land-Owner", "Pct. Med. Land-Owner",
      ↪ "Pct. Large Land-Owner",
      "Pct. Head Educ., Illiterate", "Pct. Head Educ., Lit.-Primary", "Pct. Head Educ.,
      ↪ Secondary+", "Pct. Pucca",
      "Population", "Consumption", "Gini of Consumption",
      "Percent change in consumption", "Change in Gini of consumption"
    ),
    desc = c(
      "Average linguistic distance of mother tongue from Hindi",
      "EMI exposure",
      "Estimated via NSS sample weights",
      "Average household monthly consumption expenditures (Rs.)",
      "Gini coefficient of consumption",
      "Percentage of people in an urban area",
      "Average household size",
      "Ratio of dependents (0-14, 65+) to labor force (15-64), × 100",
      "Percentage of households with a female head",
      "Percentage of Hindus",
      "Percentage of Muslims",
      "Percentage not Hindu/Muslim",
      "Scheduled Tribe",
      "Scheduled Caste",
      "Other Backward Class",
      "Owns 0.005-0.40 hectares",
      "Owns 0.41-3.00 hectares",
      "Owns $\\geq$ 3.01 hectares",
      "Percentage of household heads with educ. level: illiterate",
      "Percentage of heads with educ. level: literate-primary",
      "Percentage of heads with educ. level: above secondary",
      "Percentage in pucca (permanent) homes",
      "Estimated via NSS sample weights",
```

```
      "Average household monthly consumption expenditures (Rs.)",
      "Gini coefficient of consumption",
      "Percent change in consumption",
      "Change in the Gini coefficient of consumption"
    ),
    stringsAsFactors = FALSE
  )
  out <- public_numeric_stats(district_panel, meta, count_vars = c("npeople_0708",
  ↪  "npeople_1718"))
  out <- insert_summary_group(out, "From 2001:", "wavg_ling_degrees")
  out <- insert_summary_group(out, "From 2007-08:", "EMIE")
  out <- insert_summary_group(out, "From 2017-18:", "npeople_1718")
  out <- insert_summary_group(out, "From 2007-08 to 2017-18:", "consumption_pct_change")
  out
}
make_first_stage_table <- function(first_stage_tests, cfg = list()) {
  fs <- as.data.frame(first_stage_tests, stringsAsFactors = FALSE)
  required_cols <- c("model", "term", "estimate", "std.error", "p.value", "partial_f",
  ↪  "partial_p", "status")
  missing_cols <- setdiff(required_cols, names(fs))
  if (length(missing_cols)) {
    msg <- paste("First-stage results are missing columns:", paste(missing_cols, collapse =
  ↪  ", "))
    if (is_final_mode(cfg)) stop(msg, call. = FALSE)
    return(table_status_row("first_stage", "unavailable", msg))
  }
  if ("model" %in% names(fs) && any(fs$model %in% c("consumption", "baseline"))) fs <-
    ↪  fs[fs$model %in% c("consumption", "baseline"), , drop = FALSE]
  if (any(grepl("^[0-9]+$", as.character(fs$term)))) {
    msg <- "First-stage coefficient terms are numeric row positions; coefficient names were
  ↪  lost upstream."
    if (is_final_mode(cfg)) stop(msg, call. = FALSE)
    return(table_status_row("first_stage", "malformed", msg))
  }
  status_reasons <- unique(stats::na.omit(as.character(fs$reason[!is.na(fs$status) &
  ↪  fs$status != "estimated"])))
  fs <- fs[fs$status == "estimated" & !is.na(fs$term), , drop = FALSE]
  required_terms <- c("wavg_ling_degrees", "(Intercept)")
  missing_terms <- setdiff(required_terms, fs$term)
  if (length(missing_terms)) {
    if (length(status_reasons)) {
      msg <- paste(status_reasons, collapse = "; ")
      out <- table_status_row("first_stage", "out_of_active_pipeline", msg)
      attr(out, "table_input_failures") <- paste0("First-stage table lacks required
        ↪  coefficient(s): ", paste(missing_terms, collapse = ", "), ". Reasons: ", msg)
      return(out)
    }
    msg <- paste("First-stage table lacks required coefficient(s):", paste(missing_terms,
  ↪  collapse = ", "))
    if (is_final_mode(cfg)) stop(msg, call. = FALSE)
    return(table_status_row("first_stage", "unavailable", msg))
  }
  term_order <- c(
    "wavg_ling_degrees", "consumption_0708", "gini_cons_0708",
    "pct_urban", "avg_hh_size", "dependency_ratio", "pct_fem_head",
```

```r
    "pct_hindu", "pct_muslim", "pct_st", "pct_sc", "pct_obc",
    "pct_small_land", "pct_medium_land", "pct_large_land",
    "pct_head_lit_to_primary", "pct_head_secondary_plus", "(Intercept)"
  )
  fs <- fs[order(match(fs$term, term_order), fs$term), , drop = FALSE]
  fs <- fs[!is.na(match(fs$term, term_order)), , drop = FALSE]

  stat <- fs[fs$term == "wavg_ling_degrees", , drop = FALSE]
  f_value <- if (nrow(stat)) first_finite_value(stat, c("partial_f", "model_f")) else
↪  NA_real_
  f_p <- if (nrow(stat)) first_finite_value(stat, c("partial_p", "model_p")) else NA_real_
  f_row <- data.frame(
    Term = "Instrument's F-Statistic",
    value = paste0(sprintf("%.2f", f_value), significance_stars(f_p)),
    stringsAsFactors = FALSE
  )
  nobs_value <- first_finite_value(stat, c("nobs", "n", "N"))
  r2_value <- first_finite_value(stat, c("r.squared", "r2"))
  adj_r2_value <- first_finite_value(stat, c("adj.r.squared", "adj_r2"))
  gof <- safe_bind_rows(list(
    data.frame(Term = "Observations", value = ifelse(is.finite(nobs_value), sprintf("%.0f",
     ↪  nobs_value), ""), stringsAsFactors = FALSE),
    data.frame(Term = "R-squared", value = ifelse(is.finite(r2_value), sprintf("%.3f",
     ↪  r2_value), ""), stringsAsFactors = FALSE),
    data.frame(Term = "Adjusted R-squared", value = ifelse(is.finite(adj_r2_value),
     ↪  sprintf("%.3f", adj_r2_value), ""), stringsAsFactors = FALSE),
    f_row
  ))

  regression_display_table(
    terms = iv_table_term_label(fs$term),
    estimates = suppressWarnings(as.numeric(fs$estimate)),
    std_errors = suppressWarnings(as.numeric(fs$std.error)),
    p_values = suppressWarnings(as.numeric(fs$p.value)),
    outcome_label = "EMI Exposure",
    gof = gof
  )
}

first_finite_value <- function(df, cols) {
  for (col in cols) {
    if (!col %in% names(df)) next
    value <- suppressWarnings(as.numeric(df[[col]][[1]]))
    if (length(value) && is.finite(value)) return(value)
  }
  NA_real_
}

first_stage_table_model <- function(iv_models, district_panel) {
  model <- second_stage_table_model(iv_models)$model
  if (is.null(model) || !inherits(model, "ivreg")) return(list(model = NULL, vcov = NULL,
   ↪  add_rows = NULL))
  terms <- parse_iv_formula_terms(model)
  if (is.null(terms) || !length(terms$regressors) || !length(terms$instruments)) {
    return(list(model = NULL, vcov = NULL, add_rows = NULL))
```

```
  }
  endogenous <- setdiff(terms$regressors, terms$instruments)
  if (!length(endogenous)) endogenous <- terms$regressors[[1]]
  if (!length(endogenous) || is.na(endogenous[[1]]) || !nzchar(endogenous[[1]])) {
    return(list(model = NULL, vcov = NULL, add_rows = NULL))
  }
  formula <- stats::as.formula(paste(endogenous[[1]], "~", paste(terms$instruments, collapse
 ↪  = " + ")))
  data <- as.data.frame(district_panel)
  if (length(setdiff(all.vars(formula), names(as.data.frame(data))))) {
    return(list(model = NULL, vcov = NULL, add_rows = NULL))
  }
  fit <- tryCatch(stats::lm(formula, data = data), error = function(e) NULL)
  if (is.null(fit)) return(list(model = NULL, vcov = NULL, add_rows = NULL))
  vc <- first_stage_vcov(fit, data)
  excluded <- setdiff(terms$instruments, terms$regressors)
  excluded_term <- if (length(excluded)) excluded[[1]] else NA_character_
  wald <- first_stage_wald_test(fit, excluded_term, vc)
  add_rows <- data.frame(
    term = "Instrument's F-Statistic",
    estimate = paste0(formatC(wald$partial_f, digits = 2, format = "f"),
     ↪  significance_stars(wald$partial_p)),
    stringsAsFactors = FALSE
  )
  list(model = fit, vcov = vc, add_rows = add_rows)
}

second_stage_table_model <- function(iv_models) {
  model <- NULL
  if (is.list(iv_models) && !inherits(iv_models, c("lm", "ivreg"))) {
    hit <- intersect(c("consumption", "baseline"), names(iv_models))
    if (length(hit)) model <- iv_models[[hit[[1]]]]
  } else {
    model <- iv_models
  }
  if (is.null(model) || is_model_status_payload(model)) return(list(model = NULL, vcov =
  ↪  NULL))
  vc_fun <- clustered_model_vcov(model)
  vc <- if (is.null(vc_fun)) NULL else tryCatch(vc_fun(model), error = function(e) NULL)
  list(model = model, vcov = vc)
}

make_second_stage_table <- function(iv_models) {
  out <- tidy_iv_models(iv_models)
  out <- filter_table_model(out, c("consumption", "baseline"))
  if (!nrow(out)) return(data.frame(Term = character(), `Consumption Growth` = character(),
  ↪  check.names = FALSE))
  model <- NULL
  if (is.list(iv_models) && !inherits(iv_models, c("lm", "ivreg"))) {
    hit <- intersect(c("consumption", "baseline"), names(iv_models))
    if (length(hit)) model <- iv_models[[hit[[1]]]]
  } else {
    model <- iv_models
  }
  regression_display_table(
```

```
    terms = iv_table_term_label(out$term),
    estimates = suppressWarnings(as.numeric(out$estimate)),
    std_errors = suppressWarnings(as.numeric(out$std.error)),
    p_values = suppressWarnings(as.numeric(out$p.value)),
    outcome_label = "Consumption Growth",
    gof = model_gof_rows(model, "Consumption Growth")
  )
}

iv_table_term_label <- function(term) {
  labels <- c(
    "(Intercept)" = "Constant",
    EMIE = "EMIE",
    wavg_ling_degrees = "Linguistic distance",
    consumption_0708 = "Consumption, 2007-08",
    gini_cons_0708 = "Gini consumption, 2007-08",
    pct_urban = "Pct. urban",
    avg_hh_size = "Avg. HH size",
    dependency_ratio = "Dependency ratio",
    pct_fem_head = "Pct. female head",
    pct_hindu = "Pct. Hindu",
    pct_muslim = "Pct. Muslim",
    pct_st = "Pct. ST",
    pct_sc = "Pct. SC",
    pct_obc = "Pct. OBC",
    pct_small_land = "Pct. small land",
    pct_medium_land = "Pct. medium land",
    pct_large_land = "Pct. large land",
    pct_head_lit_to_primary = "Pct. head literate-primary",
    pct_head_secondary_plus = "Pct. head secondary+"
  )
  out <- unname(labels[term])
  out[is.na(out)] <- term[is.na(out)]
  out
}
```